VKSFT91_vSphere Kubernetes Service: Fast Track [v9.1]

Lab Manual

vmware by Broadcom

vSphere Kubernetes Service - Fast Track [v9.1]

Lab Manual

Part Number EDU-EN-VKSFT91-LAB (12-OCT-2026)

Copyright © 2026 Broadcom. All Rights Reserved. The term “Broadcom” refers to Broadcom Inc. and/or its subsidiaries. All trademarks, trade names, service marks, and logos referenced herein belong to their respective companies.

This documentation, which includes embedded help systems and electronically distributed materials (hereinafter referred to as the “Documentation”) is for your informational purposes only and is subject to change or withdrawal by Broadcom at any time.

This Documentation is proprietary information of Broadcom and may not be copied, transferred, reproduced, disclosed, modified or duplicated, in whole or in part, without the prior written consent of Broadcom, except as authorized by a valid, in effect license from Broadcom.

If you are a licensed user of the software product(s) addressed in the Documentation, you may print or otherwise make available a reasonable number of copies of the Documentation for internal use by you and your employees in connection with that software, provided that all Broadcom copyright notices and legends are affixed to each reproduced copy.

The right to print or otherwise make available copies of the Documentation is limited to the period during which the applicable license for such software remains in full force and effect. Should the license terminate for any reason, it is your responsibility to certify in writing to Broadcom that all copies and partial copies of the Documentation have been returned to Broadcom or destroyed.

TO THE EXTENT PERMITTED BY APPLICABLE LAW AND EXCEPT AS AGREED BY BROADCOM IN ITS APPLICABLE LICENSE AGREEMENT, BROADCOM PROVIDES THIS DOCUMENTATION “AS IS” WITHOUT WARRANTY OF ANY KIND, INCLUDING WITHOUT LIMITATION, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NONINFRINGEMENT. IN NO EVENT WILL BROADCOM BE LIABLE TO THE END USER OR ANY THIRD PARTY FOR ANY LOSS OR DAMAGE, DIRECT OR INDIRECT, FROM THE USE OF THIS DOCUMENTATION, INCLUDING WITHOUT LIMITATION, LOST PROFITS, LOST INVESTMENT, BUSINESS INTERRUPTION, GOODWILL, OR LOST DATA, EVEN IF BROADCOM IS EXPRESSLY ADVISED IN ADVANCE OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

Typographical Conventions

The following typographical conventions are used in this course.

Conventions	Usage and Examples

`Monospace`	Identifies command names, command options, parameters, code fragments, error messages, filenames, folder names, directory names, and path names: • Run the `esxtop` command. • ... found in the `/var/log/messages` file.
`Monospace Bold`	Identifies user inputs: • Enter **`ipconfig /release`**.
Boldface	Identifies user interface controls: • Click the **Configuration** tab.
Italic	Identifies book titles: • *vSphere Virtual Machine Administration*
< >	Indicates placeholder variables: • <ESX_host_name> • ... the `Settings/<Your_Name>.txt` file

Contents

Lab 1 Cilium CNI Integration with VKS 1

Task 1: Verify Add-on Repository Prerequisites 1

Task 2: Configure AddonInstall and AddonConfig Resources 1

Task 3: Deploy VKS Cluster with Cilium CNI 2

Task 4: Verify Cilium eBPF Deployment Status 2

Lab 2 Star Wars App Deployment and L3–L7 Policy Enforcement with Cilium 4

Task 1: Connect to the VKS Cluster and Deploy Demo Workloads 4

Task 2: Enforce L3/L4 Identity-Based Security Policy 5

Task 3: Visualize Allowed and Dropped L3/L4 Traffic in Hubble UI 5

Task 4: Apply L7 Policy and Enable Envoy Proxy Filtering 6

Task 5: Visualize Permitted and Blocked L7 HTTP Traffic in Hubble UI 6

Lab 3 Deploying Multi-NIC VKS Cluster and installing Multus and Whereabouts 8

Task 1: Create Calico AddonConfig and Secondary Subnet 8

Task 2: Deploy Multi-NIC VKS Cluster 9

Task 3: Install and Configure Multus CNI Add-on 9

Task 4: Install and Configure Whereabouts IPAM Add-on 10

Task 5: Verify Deployment Status 10

Lab 4 Deploying Multi-NIC Pods with Multus and Whereabouts 12

Task 1: Deploy Dual-Interface Application Workload 12

Task 2: Verify Pod Interfaces and Traffic Routing Across Secondary Network 13

Lab 5 Installing Grafana Add-on on VKS Cluster and Integrating with Prometheus 14
Task 1: Configure Grafana AddonRepository and AddonRepositoryInstall 14
Task 2: Configure and Deploy Grafana via AddonConfig and AddonInstall 16
Task 3: Enable Prometheus Server on Target Cluster 17
Task 4: Integrate Prometheus Data Source and Dashboard in Grafana 18
Lab 6 Installing Headlamp on VKS Cluster 20
Task 1: Install Headlamp Add-on via VCF Automation 20
Task 2: Generate Bearer Authentication Token and Access Headlamp UI 21
Lab 7 Deploying vSphere Supervisor 23
Task 1: Verify Virtual Network Appliance Cluster 23
Task 2: Update the VPC Connectivity profile to assign the VNA Cluster 24
Task 3: Create External Connection 25
Task 4: Create vSphere Zone 26
Task 5: Create VM Storage Policy 26
Task 6: Activate Supervisor 27
Task 7: Verify the status of the Supervisor 28
Lab 8 Exposing Workloads with Contour Ingress Controller 31
Task 1: Connect to VKS Cluster using VKS Cluster Management 31
Task 2: Installing cert-manager 32
Task 3: Installing Contour 32
Task 4: Configure Ingress Routing via Contour HTTPProxy 33
Task 5: Deploy Authenticated NGINX Workload with Harbor Credentials 34
Lab 9 Deploying Containers Using VCF Automation 35
Task 1: Deploy a Container using the Container Service 35
Task 2: Verify the Container Details 36
Task 3: Scale up the Container 37
Task 4: Verify the Pod Details in vCenter 37
Lab 10 Deploying MySQL DB VM in Tenant Organization 39
Task 1: Log In to the VCF Automation Organization Portal 39
Task 2: Deploy MySQL DB Virtual Machine 39
Lab 11 Deploying a VKS Cluster in Tenant Organization 42

Task 1: Log In to the VCF Automation Organization Portal 42
Task 2: Deploy VKS Cluster 42
Task 3: Verify the VKS Cluster Configuration 44
Lab 12 Connecting to VKS Cluster using SSH 46
Task 1: Fetch VKS cluster SSH password and Node IP address 46
Task 2: Create a Jumpserver to connect to the VKS cluster 47
Lab 13 Deploying OpenCart Application 50
Task 1: Deploy OpenCart Application 50
Lab 14 Deploying Applications using Helm 53
Task 1: Upload Images and Helm Chart to Harbor. 53
Task 2: Deploy Nginx using Helm 56
Lab 15 Upgrading and Rolling Back Applications using Helm 58
Task 1: Upload New Helm Chart to Harbor 58
Task 2: Upgrade Nginx using Helm 59
Task 3: Rollback to a Previous Revision 60
Lab 16 Create multi-nic nodes on a VKS Cluster 61
Task 1: Create customised SubnetSet for Primary Network 61
Task 2: Create Parent Subnet 62
Task 3: Create Child Overlay Subnet 62
Task 4: Create Child Overlay Subnet to Parent Subnet Binding 63
Task 5: Create a Distributed VLAN connection from VCF Automation Provider Management Portal 63
Task 6: Create VLAN extension subnet from VCF Automation Org Portal 64
Task 7: Create Child VLAN Extension Subnet to Parent Subnet Binding 65
Task 8: Reserve an IP range for the pod from the Child Subnets 66
Task 9: Enable featureGates in Antrea configuration 66
Task 10: Create a multi-nic cluster 66
Task 11: Verify the interfaces on VKS cluster nodes 67
Lab 17 Creating multi-nic pods on a multi-nic VKS Cluster 69
Task 1: Export the vks-multinic01 kubeconfig 69
Task 2: Create an IPPool custom resource 69
Task 3: Create a NetworkAttachmentDefinition custom resource 70
Task 4: Deploy Pods with multiple secondary networks 70

Lab 18 Enabling AppArmor Profile on VKS Cluster nodes 72
Task 1: Create an AppArmor profile 72
Task 2: Update the VKS cluster with the apparmor profile 73
Task 3: Verify the AppArmor profile on VKS cluster nodes 73
Lab 19 Enabling TuneD profile on VKS Cluster nodes 75
Task 1: Create a custom TuneD profile 75
Task 2: Update the VKS cluster with the custom TuneD profile 76
Task 3: Verify the custom TuneD profile on VKS cluster nodes 76
Lab 20 Monitoring VKS Clusters from VCF Operations 78
Task 1: Verify Metrics Aggregator Service status 78
Task 2: Verify Monitoring status on VKS cluster 79
Task 3: Verify Prometheus and Telegraf Add-ons status 79
Task 4: Monitoring VKS Cluster from VCF Operations 80
Task 5: Monitoring VKS Cluster from VCF Automation 81
Task 6: Monitoring VKS Cluster from VKS Cluster Management 81
Lab 21 Backing Up Applications Using Velero 83
Task 1: Enable Data Protection on the Cluster 83
Task 2: Configure S3 Account Credentials 84
Task 3: Map the S3 Target Backup Location 84
Task 4: Perform Backup Prechecks 85
Task 5: Create a Targeted Namespace Backup 86
Task 6: Perform Backup Postchecks 87
Lab 22 Restoring Applications Using Velero 88
Task 1: Simulate an Application Failure 88
Task 2: Execute an Application Restore 89
Task 3: Verify Application Recovery 90
Lab 23 Updating a VKS Cluster using a Custom Rolling Update Strategy 91
Task 1: Customize VKS Cluster Rolling Update Strategy 91
Task 2: Registering a New Version of the VKS 93
Task 3: Upgrading the VKS Version 94
Task 4: Updating the VKS Cluster Using VCF CLI 94
Answer Key 98

Lab 1 Cilium CNI Integration with VKS

Objective and Tasks

Provision a VKS cluster configured with the Cilium CNI add-on running in eBPF direct routing mode without kube-proxy.

1. Verify Add-on Repository Prerequisites
2. Configure AddonInstall and AddonConfig Resources
3. Deploy VKS Cluster with Cilium CNI
4. Verify Cilium eBPF Deployment Status

Task 1: Verify Add-on Repository Prerequisites

1. Open a new **Terminal** from taskbar
2. Switch to the Supervisor context

   `vcf context use sup-wld`

   At **Password** prompt, enter **`VMware123!VMware123!`**
3. Verify that the VKS add-on repository version **3.7.0+20260618** or later is available:

   `kubectl get addonrepository -n vmware-system-vks-public`
4. Retrieve and record the name of the vSphere Namespace you will use in your manifest files.

   `kubectl get ns | grep acme-prod`

Task 2: Configure AddonInstall and AddonConfig Resources

1. Review the AddonInstall manifest at `~/Lab-files/vks-3.7/cilium/addoninstall-cilium-no-kubeproxy.yaml`

Ensure the vSphere Namespace matches your environment.

2. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/cilium/addoninstall-cilium-no-kubeproxy.yaml

3. Verify that the **addoninstall** resource has been created.

   kubectl get addoninstall -n acme-prod-xxxx | grep vks-cilium01

4. Review the AddonConfig manifest at `~/Lab-files/vks-3.7/cilium/addonconfig-cilium-no-kubeproxy.yaml`

   Ensure the vSphere Namespace matches your environment.

5. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/cilium/addonconfig-cilium-no-kubeproxy.yaml

6. Verify that the **addonconfig** resource has been created.

   kubectl get addonconfig -n acme-prod-xxxx | grep vks-cilium01-cilium

Task 3: Deploy VKS Cluster with Cilium CNI

1. Review the cluster manifest at `~/Lab-files/vks-3.7/cilium/vks-cilium01.yaml`

   Ensure the vSphere Namespace matches your environment.

2. Provision the cluster

   kubectl apply -f ~/Lab-files/vks-3.7/cilium/vks-cilium01.yaml

3. Verify cluster provisioning has initiated

   kubectl get cluster vks-cilium01 -n acme-prod-xxxx -w

4. Wait until the status displays **AVAILABLE** as `TRUE` and the **PHASE** updates to `Provisioned`. Press Ctrl+C to exit.

Task 4: Verify Cilium eBPF Deployment Status

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**

   a. Under Organization name, enter **acme-org**

b. Log in to the VCF Automation Organization Portal.

- Log in with local account
- **Username**: **admin**
- **Password**: **VMware123!VMware123!**

2. In the Organization portal, click on **Build & Deploy** tab.
3. On **Kubernetes** Page under **Services**, click **vks-cilium01**
4. Click **DOWNLOAD KUBECONFIG FILE.** This Kubeconfig is used to connect to the VKS Cluster
5. Click **Save**
6. Switch to **Termimal**
7. Export the downloaded Kubeconfig file

   export KUBECONFIG=/home/holuser/Downloads/vks-cilium01-kubeconfig.yaml

   a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

      kubectl get nodes

      Above command should list **vks-cilium01** nodes

8. Verify Cilium pods are in Running status:

   kubectl get pods -n kube-system -l k8s-app=cilium -o wide

9. Verify the status of hubble

   kubectl get pods,svc -n kube-system | grep hubble

10. Validate eBPF direct routing and KubeProxyReplacement status:

    kubectl exec -n kube-system ds/cilium -- cilium status

    Confirm **KubeProxyReplacement** is True

    Confirm **Hubble** and **IPAM** services report Ok

    Cilium 0/1 node status is normal in 2-node setups, agents only probe the 1 remote peer.

Lab 2 Star Wars App Deployment and L3–L7 Policy Enforcement with Cilium

Objective and Tasks

In this lab, you deploy the Star Wars demo application on a Cilium-enabled VKS cluster and enforce defense-in-depth security policies across eBPF network identities (L3/L4) and Envoy HTTP inspection (L7).

1. Connect to the VKS Cluster and Deploy Demo Workloads
2. Enforce L3/L4 Identity-Based Security Policy
3. Visualize Allowed and Dropped L3/L4 Traffic in Hubble UI
4. Apply L7 HTTP Policy and Envoy Proxy Filtering
5. Visualize Permitted and Blocked L7 HTTP Traffic in Hubble UI

Task 1: Connect to the VKS Cluster and Deploy Demo Workloads

1. Open a new **Terminal** and export the **KUBECONFIG** for the Cilium cluster

   ```
   export KUBECONFIG=/home/holuser/Downloads/vks-cilium01-kubeconfig.yaml
   ```

2. Create the target namespace

   ```
   kubectl create namespace cilium-starwars
   ```

3. Label the namespace:

   ```
   kubectl label ns cilium-starwars pod-security.kubernetes.io/enforce=privileged --overwrite
   ```

4. Review the application manifest at `~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/http-sw-app.yaml`
5. Deploy the application workloads:

   `kubectl apply -f ~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/http-sw-app.yaml`
6. Verify that all pods are running and the ClusterIP is assigned to deathstar.

   `kubectl get pods,svc -n cilium-starwars`
7. Note the endpoint details for the deathstar service.

   `kubectl describe svc -n cilium-starwars deathstar`

   Rules will be applied to these endpoints.

Task 2: Enforce L3/L4 Identity-Based Security Policy

1. Review the L3/L4 policy manifest at `~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/sw_l3_l4_policy.yaml`
2. Apply the L3/L4 network policy:

   `kubectl apply -f ~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/sw_l3_l4_policy.yaml`
3. Locate the active Cilium pod running on worker node-pool-1

   `kubectl get pods -n kube-system -l k8s-app=cilium -o wide | grep node-pool-1`
4. Confirm that ingress policy enforcement is active for deathstar endpoints via cilium-dbg:

   `kubectl -n kube-system exec cilium-bvtwp -- cilium-dbg endpoint list | grep -E 'ENDPOINT|deathstar'`

   Confirm POLICY (ingress) shows Enabled for deathstar endpoints.

Task 3: Visualize Allowed and Dropped L3/L4 Traffic in Hubble UI

1. Execute an authorized HTTP POST request from tiefighter (org: empire):

   `kubectl exec -n cilium-starwars tiefighter -- curl -s -XPOST deathstar.cilium-starwars.svc.cluster.local/v1/request-landing`

   Verify the response output displays **Ship landed**.
2. Open Google Chrome and access the Hubble UI using `http://<Hubble UI LoadBalancer IP>`

Retrieve the Hubble UI LoadBalancer IP by running:

kubectl get svc -n kube-system hubble-ui

3. Select the **cilium-starwars** namespace from the dropdown menu.

   Confirm that traffic from **tiefighter** to **deathstar** on port 80 reports verdict **forwarded**

4. Execute an unauthorized request from xwing (org: alliance):

   kubectl exec -n cilium-starwars xwing -- curl -s --connect-timeout 5 -XPOST deathstar.cilium-starwars.svc.cluster.local/v1/request-landing

   Confirm the request times out with **command terminated with exit code 28**

5. Observe the Hubble UI service map:

   Confirm traffic from xwing to deathstar is dropped at the eBPF layer and reports verdict **dropped**.

Task 4: Apply L7 Policy and Enable Envoy Proxy Filtering

1. Delete the existing L3/L4 policy:

   kubectl delete -f ~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/sw_l3_l4_policy.yaml

2. Review the L7 policy manifest at `~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/sw_l7_policy.yaml.`

3. Apply the L7 policy manifest

   kubectl apply -f ~/Lab-files/vks-3.7/cilium/cilium-starwars-demo/sw_l7_policy.yaml

4. Verify active L7 ingress policy enforcement on deathstar endpoints via cilium-dbg:

   kubectl -n kube-system exec cilium-bvtwp -- cilium-dbg endpoint list | grep -E 'ENDPOINT|deathstar'

   Confirm POLICY (ingress) shows Enabled.

Task 5: Visualize Permitted and Blocked L7 HTTP Traffic in Hubble UI

1. Test an authorized HTTP POST /v1/request-landing call from tiefighter:

   kubectl exec -n cilium-starwars tiefighter -- curl -s -i -XPOST deathstar.cilium-starwars.svc.cluster.local/v1/request-landing

Verify response headers show **HTTP/1.1 200 OK**, **server: envoy**, and body text **Ship landed**.

2. Check Hubble UI to inspect the L7 flow:

   Confirm L7 info displays POST /v1/request-landing 0ms with verdict forwarded.

3. Test an unauthorized HTTP PUT /v1/exhaust-port call from tiefighter:

   `kubectl exec -n cilium-starwars tiefighter -- curl -s -i -XPUT deathstar.cilium-starwars.svc.cluster.local/v1/exhaust-port`

   Verify response headers show **HTTP/1.1 403 Forbidden**, **server: envoy**, and body text **Access denied**

4. Check Hubble UI to confirm Envoy-enforced L7 drops:

   Confirm L7 info displays **PUT /v1/exhaust-port 0ms** with verdict **dropped**

Lab 3 Deploying Multi-NIC VKS Cluster and installing Multus and Whereabouts

Objective and Tasks

Provision a secondary network infrastructure, deploy a Multi-NIC VKS cluster with Calico CNI, and configure the Multus CNI and Whereabouts IPAM add-ons.

1. Create Calico AddonConfig and Secondary Subnet
2. Deploy Multi-NIC VKS Cluster
3. Install and Configure Multus CNI Add-on
4. Install and Configure Whereabouts IPAM Add-on

Task 1: Create Calico AddonConfig and Secondary Subnet

1. Open a new **Terminal** from the taskbar
2. Switch to the Supervisor context

   `vcf context use sup-wld`

   At **Password** prompt, enter **`VMware123!VMware123!`**
3. Retrieve and record the name of the vSphere Namespace you will use in your manifest files.

   `kubectl get ns | grep acme-prod`
4. Create calico-addonconfig.yaml to pin Calico network autodetection to eth0. Review the AddonConfig manifest at `~/Lab-files/vks-3.7/multus/calico-addonconfig.yaml`

   Ensure the vSphere Namespace matches your environment.

5. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/multus/calico-addonconfig.yaml

6. Create secondary-subnet.yaml for the secondary node network vNIC. Review the manifest at `~/Lab-files/vks-3.7/multus/secondary-subnet.yaml`

   Ensure the vSphere Namespace matches your environment.

7. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/multus/secondary-subnet.yaml

Task 2: Deploy Multi-NIC VKS Cluster

1. Review the cluster manifest at `~/Lab-files/vks-3.7/multus/vks-multus01.yaml` specifying eth1 network mapping:

   Ensure the vSphere Namespace matches your environment.

2. Provision the cluster

   kubectl apply -f ~/Lab-files/vks-3.7/multus/vks-multus01.yaml

3. Verify cluster provisioning has initiated

   kubectl get cluster vks-multus01 -n acme-prod-xxxx -w

4. Wait until the status displays **AVAILABLE** as `TRUE` and the **PHASE** updates to `Provisioned`. Press Ctrl+C to exit.

Task 3: Install and Configure Multus CNI Add-on

1. Review the AddonInstall manifest at `~/Lab-files/vks-3.7/multus/multus-addoninstall.yaml`

   Ensure the vSphere Namespace matches your environment.

2. Apply the installation manifest

   kubectl apply -f ~/Lab-files/vks-3.7/multus/multus-addoninstall.yaml

3. Create multus-addonconfig.yaml to define resource limits and secondary IP pools. Review the manifest at `~/Lab-files/vks-3.7/multus/multus-addonconfig.yaml`

   Ensure the vSphere Namespace matches your environment.

4. Apply the configuration manifest

   kubectl apply -f ~/Lab-files/vks-3.7/multus/multus-addonconfig.yaml

Task 4: Install and Configure Whereabouts IPAM Add-on

1. Review the AddonInstall manifest at `~/Lab-files/vks-3.7/multus/whereabouts-addoninstall.yaml`

   Ensure the vSphere Namespace matches your environment.

2. Apply the installation manifest

   kubectl apply -f ~/Lab-files/vks-3.7/multus/whereabouts-addoninstall.yaml

3. Create `whereabouts-addonconfig.yaml` to define resource limits and secondary IP pools. Review the manifest at `~/Lab-files/vks-3.7/multus/multus-addonconfig.yaml`

   Ensure the vSphere Namespace matches your environment.

4. Apply the configuration manifest

   kubectl apply -f ~/Lab-files/vks-3.7/multus/whereabouts-addonconfig.yaml

Task 5: Verify Deployment Status

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **acme-org**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - **Username**: **admin**
      - **Password**: **VMware123!VMware123!**
2. In the Organization portal, click on **Build & Deploy** tab.
3. On **Kubernetes** Page under **Services**, click **vks-multus01**
4. Click **DOWNLOAD KUBECONFIG FILE.** This Kubeconfig is used to connect to the VKS Cluster
5. Click **Save**
6. Switch to **Terminal**

7. Export the downloaded Kubeconfig file

   export KUBECONFIG=/home/holuser/Downloads/vks-multus01-kubeconfig.yaml

   a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

      kubectl get nodes

      Above command should list **vks-multus01** nodes

8. Verify that the Calico node pods and controller are running on all nodes:

   kubectl get pods -n kube-system -l k8s-app=calico-node -o wide

   kubectl get pods -n kube-system -l k8s-app=calico-kube-controllers

9. Verify Multus is running on every node:

   kubectl get daemonset -n multus-cni

   kubectl get pods -n multus-cni -o wide

10. Verify that the Whereabouts pods are running on every node:

    kubectl get daemonset -n whereabouts-cni

    kubectl get pods -n whereabouts-cni -o wide

Lab 4 Deploying Multi-NIC Pods with Multus and Whereabouts

Objective and Tasks

Deploy a dual-interface workload pod attached to both the primary Calico network and secondary Multus network, then verify routing and end-to-end connectivity.

1. Deploy Dual-Interface Application Workload
2. Verify Pod Interfaces and Traffic Routing Across Secondary Network

Task 1: Deploy Dual-Interface Application Workload

1. Export the downloaded Kubeconfig file

   `export KUBECONFIG=/home/holuser/Downloads/vks-multus01-kubeconfig.yaml`

   a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

      `kubectl get nodes`

      Above command should list **vks-multus01** nodes

2. Review the workload manifest at `~/Lab-files/vks-3.7/multus/multus-workload.yaml`
3. Deploy the application

   `kubectl apply -f ~/Lab-files/vks-3.7/multus/multus-workload.yaml`

Task 2: Verify Pod Interfaces and Traffic Routing Across Secondary Network

1. Verify pods are running in multus-test

   `kubectl get pods -n multus-test`

2. Exec into a running pod and inspect network interfaces:

   `kubectl exec -it -n multus-test <pod-name> -- ip a`

   Confirm **eth0** displays an IP assigned via Calico CNI from Pod CIDR 192.0.2.0/16

   Confirm **net1** displays an IP assigned via Multus/Whereabouts from ipam range 172.30.0.98 - 172.30.0.110

3. Inspect the IP routing table inside the pod

   `kubectl exec -it -n multus-test <pod-name> -- route`

4. Ping the secondary network gateway through **net1**

   `kubectl exec -it -n multus-test <pod-name> -- ping 10.1.1.1 -c 1`

   Verify 0% packet loss to validate secondary NIC routing

Lab 5 Installing Grafana Add-on on VKS Cluster and Integrating with Prometheus

Objective and Tasks

Deploy the Grafana add-on using Helm-based Addon CRs, enable the Prometheus server component on the target VKS cluster, and integrate Prometheus with Grafana for node monitoring.

1. Configure Grafana AddonRepository and AddonRepositoryInstall
2. Configure and Deploy Grafana via AddonConfig and AddonInstall
3. Enable Prometheus Server on Target Cluster
4. Integrate Prometheus Data Source and Dashboard in Grafana

Task 1: Configure Grafana AddonRepository and AddonRepositoryInstall

1. Open a new **Terminal** from the taskbar
2. Switch to the Supervisor context

   `vcf context use sup-wld`

   At **Password** prompt, enter **`VMware123!VMware123!`**
3. Retrieve and record the name of the vSphere Namespace you will use in your manifest files.

   `kubectl get ns | grep acme-prod`
4. Review the AddonRepository manifest at `~/Lab-files/vks-3.7/grafana/addon_repository.yaml`

Ensure the vSphere Namespace matches your environment.

5. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/grafana/addon_repository.yaml

6. Review the AddonRepositoryInstall manifest at `~/Lab-files/vks-3.7/grafana/addon_repository_install.yaml`

   Ensure the vSphere Namespace matches your environment.

7. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/grafana/addon_repository_install.yaml

8. Verify that AddonRepositoryInstall status reports `READY: True`

   kubectl get addonrepositoryinstall -n vmware-system-vks-public grafana-repo-install

   Expected Output:

```
vmware-system-vks-public   grafana-repo-install
grafana-repo
1.0.0            True    grafana-repo                 9m1s
```

   Note: If the status is READY: True, skip proxy settings and move directly to Task 2

9. If the public Grafana repository is unreachable due to proxy restricted internet access, the AddonRepositoryInstall resource will report `READY: False`

   Configure proxy settings on the tanzu-addons-controller-manager deployment. This configuration is non-persistent and will reset upon restart or reconciliation.

10. Open a new **Terminal** and SSH to the Workload vCenter

    ssh root@vc-wld01-a.site-a.vcf.lab

11. Run the below decrypt script to print the vSphere Supervisor management Floating IP (FIP) and the root password

    a. **/usr/lib/vmware-wcp/decryptK8Pwd.py**

    b. Sample Output:
    Cluster: domain-c9:6dac4cec-9913-470e-8faa-f078f982e937
    IP: 10.1.1.188
    PWD: [FoKD-*?X'7S,>hp

    Enter **exit** to logout of the vCenter

12. Log in to the vSphere Supervisor by connecting to the Supervisor management FIP using the root username and password

ssh root@<Supervisor-Mgmt-FIP>

Enter the password retrieved in the last step

13. Appy the proxy settings

```
kubectl set env deployment/tanzu-addons-controller-manager -
n svc-tkg-642rf \
  HTTP_PROXY="http://proxy.site-a.vcf.lab:3128" \
  HTTPS_PROXY="http://proxy.site-a.vcf.lab:3128" \

NO_PROXY="localhost,127.0.0.1,cluster.local,svc,vcf.lab,10.0
.0.0/8,10.1.1.0/24,10.100.0.0/14,172.16.0.0/12,172.29.0.0/16
,172.30.0.0/16,172.31.0.0/16,192.168.0.0/16,198.18.0.0/16,do
cker-registry.kube-system.svc"
```

Expected Output: `deployment.apps/tanzu-addons-controller-manager env updated`

14. Return to your other terminal session and check the AddonRepositoryInstall resource to confirm its status has updated to `READY: True`

Task 2: Configure and Deploy Grafana via AddonConfig and AddonInstall

1. Review the AddonConfig manifest at `~/Lab-files/vks-3.7/grafana/addon_config.yaml`

   Ensure the vSphere Namespace matches your environment.

2. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/grafana/addon_config.yaml

3. Review the AddonInstall manifest at `~/Lab-files/vks-3.7/grafana/addon_install.yaml`

   Ensure the vSphere Namespace matches your environment.

   Verify that the addon is targeted to install on the vks-mgmt01 cluster.

   `cluster.x-k8s.io/cluster-name: vks-mgmt01`

4. Apply the manifest file.

   kubectl apply -f ~/Lab-files/vks-3.7/grafana/addon_install.yaml

5. Switch KUBECONFIG context to the target cluster **vks-mgmt01**

```
export KUBECONFIG=/home/holuser/Downloads/vks-mgmt01-kubeconfig.yaml
```

6. Verify the Helm release status

   ```
   kubectl get helmrelease -n vmware-system-helm
   ```

   Helm release is marked `Ready: True`

   Note: If the status is READY: True, skip proxy settings and move directly to next step.

   a. If the Helm release status reports `Ready: False` due to proxy restricted internet access, apply the proxy configuration to the source-controller pod. Note that this configuration is non-persistent and will reset upon restart or reconciliation.

   ```
   kubectl set env deployment/source-controller -n vmware-system-helm \
     http_proxy="http://10.1.10.129:3128" \
     https_proxy="http://10.1.10.129:3128" \
     NO_PROXY="127.0.0.1,localhost,source-controller,helm-controller,10.0.0.0/8,172.16.0.0/12,192.168.0.0/16,198.48.0.0/16,.svc,.cluster.local"
   ```

   Expected Output: `deployment.apps/source-controller env updated`

   Verify that the source-controller pod restarts and returns to the Running state.

   ```
   kubectl get pods -n vmware-system-helm
   ```

   Wait until the Helm release status reports `Ready: True`

   ```
   kubectl get helmrelease -n vmware-system-helm
   ```

7. Verify the pod status and get the LoadBalancer IP

   ```
   kubectl get pods,svc -n grafana-system
   ```

8. Fetch the admin password

   ```
   kubectl get secret --namespace grafana-system grafana-system-vks-mgmt01-grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
   ```

Task 3: Enable Prometheus Server on Target Cluster

Enable the Prometheus Server on the **vks-multus01** cluster.

1. Edit the Prometheus AddonConfig on the Supervisor cluster to set prometheus: true, define storageClassName, and configure the service settings.

kubectl edit addonconfig -n acme-prod-5zn65 vks-multus01-prometheus

Update the spec as below:

```
spec:
  addonConfigDefinitionRef:
    name: prometheus.kubernetes.vmware.com.3.5.3---vmware.1-vks.1
    namespace: vmware-system-vks-public
  clusterName: vks-multus01
  values:
    deploycomponents:
      alertmanager: false
      kube-state-metrics: true
      node-exporter: true
      prometheus: true
      pushgateway: false
    prometheus:
      pvc:
        storageClassName: wld-storage-policy
      service:
        port: 80
        targetPort: 9090
        type: LoadBalancer
```

2. Verify that Prometheus server is deployment on vks-multus01 cluster

   Export the KUBECONFIG

   export KUBECONFIG=/home/holuser/Downloads/vks-multus01-kubeconfig.yaml

   kubectl get pods,svc -n tanzu-system-monitoring

3. Note the assigned **EXTERNAL-IP** for service/prometheus-server

Task 4: Integrate Prometheus Data Source and Dashboard in Grafana

1. Open Google Chrome and log into Grafana using the LoadBalancer IP.

   http://<LoadBalancer IP>

   Enter admin username and admin password

2. Navigate to **Connections** > **Add new connection**, enter **Prometheus** in the search bar, select **Prometheus**, and click **Add new data source**.

3. Set **Name** to **`vks-multus01-prometheus`** and **Prometheus server URL** to the **`Load Balancer IP`**
4. Click **Save & test**
5. Navigate to **Dashboards** > **New** > **Import**, enter dashboard ID **315** in **Grafana.com dashboard URL or ID** bar, and click Load to view **Kubernetes cluster metrics via Prometheus** field updated.

   Select **vks-multus01-prometheus** as the data source, and click **Import**.
6. Verify that metrics are populating on the dashboard. Ensure the **Node** dropdown displays all cluster nodes.

Lab 6 Installing Headlamp on VKS Cluster

Objective and Tasks

Install the Headlamp add-on via VCF Automation UI, configure RBAC service account credentials, and log into the Headlamp dashboard.

1. Install Headlamp Add-on via VCF Automation
2. Generate Bearer Authentication Token and Access Headlamp UI

Task 1: Install Headlamp Add-on via VCF Automation

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - User name: **`admin`**
      - Password: **`VMware123!VMware123!`**
2. Click the **Manage & Govern** tab
3. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
4. Click on VKS cluster **vks-mgmt01** and navigate to **Add-ons**
5. Under **Repositories**, verify that the below default Repository is present.

   Name: default-addon-repo-install-3.7.0-20260618

   URL: depot.kube-system.svc/vcf/vks-standard-packages/ga/3.7.0-20260618/vks-addons:3.7.0-20260618

Status: Succeeded

6. Within the **Add-ons** tab, under **Available Add-ons** section, from **headlamp** card, click **INSTALL ADD-ON**
7. In **Package name** field, enter **vks-mgmt01-headlamp**
8. In **Package version** field, select **headlamp.kubernetes.vmware.com.0.42.0-vmware.1-vks.1**
9. Under **Package Configuration**, no need to add any data-values yaml in **Overlay YAML** tab for headlamp.
10. Click **INSTALL ADD-ON** on top right
11. Under **Installed Add-ons** section, a card gets created for the Add-on **vks-mgmt01-headlamp**

    Wait for the package to reconcile and status to become green.

Task 2: Generate Bearer Authentication Token and Access Headlamp UI

1. Switch to **Terminal**
2. Create a content using VCF CLI for **vks-mgmt01** cluster

   vcf context create vksmgmt01 --endpoint=<Supervisor VIP> --insecure-skip-tls-verify --workload-cluster-name=vks-mgmt01 --workload-cluster-namespace=acme-prod-xxxx

   Example - **vcf context create vksmgmt01 --endpoint=10.100.2.4 --insecure-skip-tls-verify --workload-cluster-name=vks-mgmt01 --workload-cluster-namespace=acme-prod-5zn65**
3. Switch to the cluster context

   kubectl config use-context vksmgmt01:vks-mgmt01
4. Review headlamp-admin.yaml at `~/Lab-files/vks-3.7/headlamp/headlamp-admin.yaml`
5. Apply RBAC resources

   `kubectl apply -f ~/Lab-files/vks-3.7/headlamp/headlamp-admin.yaml`
6. Generate the service account token for authentication

   kubectl create token headlamp-admin -n kube-system
7. Copy the token string output

8. Retrieve the Headlamp LoadBalancer IP address

   `kubectl get svc -n headlamp | grep LoadBalancer`

9. Open Google Chrome and navigate to **`https://<headlamp LoadBalancer IP>`**
10. Paste the generated bearer token into the authentication screen to login.
11. Confirm access to the Headlamp Workloads overview page.

Lab 7 Deploying vSphere Supervisor

Objective and Tasks

You perform the below tasks:

1. Verify Virtual Network Appliance Cluster
2. Update the VPC Connectivity profile to assign the VNA Cluster
3. Create External Connection
4. Create vSphere Zone
5. Create VM Storage Policy
6. Activate Supervisor
7. Verify the status of the Supervisor

Task 1: Verify Virtual Network Appliance Cluster

You log in to the **vc-mgmt-a.site-a.vcf.lab** vCenter to verify the VNA Cluster

1. Log in to **vc-mgmt-a.site-a.vcf.lab** as **administrator@vsphere.local** using the vSphere Client.
   a. From your desktop, open **Firefox** browser. From the bookmarks toolbar, click **Region A > vc-mgmt-a** to open vSphere Client

      If the You connection is not private message appears, click **Advanced > Proceed to vc-mgmt-a.site-a.vcf.lab (unsafe) link**.

   b. Log in by using the saved username and password.

      Username: **`administrator@vsphere.local`**

      Password: **`VMware123!VMware123!`**

      The vSphere Client opens and the home page appears.

2. From the vSphere Client main menu, select **Inventory**.
3. In the **Hosts and Clusters** pane, select **vc-mgmt-a.site-a.vcf.lab**.
4. Click **Configure** tab.
5. Expand **Networking** and select **VNA Clusters**
6. Click **>** next to **vna-cluster-mgmt-a** to see the VNA cluster details.

   Verify that 2 nodes are listed and below is the status for both the nodes.

- The **Connectivity Status** is **Up**

- The **Status** is **Success**

Task 2: Update the VPC Connectivity profile to assign the VNA Cluster

You update the default VPC Connectivity profile to assign the VNA Cluster in **vc-mgmt-a.site-a.vcf.lab** vCenter

1. In vSphere Client, for **vc-mgmt-a.site-a.vcf.lab** under **Inventory**, go to **Networking** pane. Expand **dc-a** as your Datacenter, click **Virtual Private Clouds > Configure >** expand **Settings > IP Blocks**

   Click **ADD IP ADDRESS BLOCK**

   Update the following fields as per the table:

Parameter	Value
Name	DTGW_External_IP_Block-01
Visibility	External
CIDR/Ranges	10.1.7.128/25

   Click **SAVE**

   Verify that the IP Block **DTGW_External_IP_Block-01** status is **Success**

2. In vSphere Client, for **vc-mgmt-a.site-a.vcf.lab**, under **Inventory**, go to **Networking** pane. Click **Virtual Private Clouds > Configure > Settings > Connectivity Profiles**.
3. Click vertical dotted line next to **Default VPC Connectivity Profile** and click **Edit**

   Update the following fields as per the table:

Note: These values are available for selection from a drop-down menu, when you click in the particular field.

Parameter	Value
External IP Blocks	DTGW_External_IP_Block-01
Virtual Network Appliance Cluster	vna-cluster-mgmt-a
N-S Services	On
Default Outbound NAT	On
External IP Block for Default Outbound NAT	DTGW_External_IP_Block-01

Click **SAVE**

Verify that the **Default VPC Connectivity Profile** status is **Success**

Task 3: Create External Connection

You create a distributed external connection and attach it to the default Transit Gateway.

1. In the **Hosts and Clusters** pane, select **vc-mgmt-a.site-a.vcf.lab**.
2. Click **Configure** tab.
3. Expand **Networking** and select External Connection
4. Click **ADD EXTERNAL CONNECTION**

   Update the following fields as per the table:

Parameter	Value
Name	MGMT-DTGW01-External-Connection
Type	Distributed VLAN Connection (Uncheck Dedicated to Subnet)
VLAN ID	22
Gateway CIDR IPv4 Address	10.1.7.129/25

   Click **SAVE**

5. Attach the distributed external connection to the default Transit Gateway

a. From **Networking** pane, expand **dc-a** as your Datacenter, click **Virtual Private Clouds > Networks > Transit Gateways**

b. Click the vertical dotted line next to **Default Transit Gateway**

c. Click **EDIT**

6. On the **Connection** drop-down, select **Distributed VLAN**

   Click **YES**

7. On the **External Connection** drop-down, select **MGMT-DTGW01-External-Connection**
8. Click **SAVE**
9. Verify that **Default Transit Gateway** has got the **External Connection** set to **MGMT-DTGW01-External-Connection** and status is **Success**

Task 4: Create vSphere Zone

You create the vSphere zone in **vc-mgmt-a.site-a.vcf.lab** vCenter

1. In vSphere Client, for **vc-mgmt-a.site-a.vcf.lab**, under **Inventory**, go to **Hosts and Clusters** pane.
2. Click **vc-mgmt-a.site-a.vcf.lab > Configure > vSphere Zones**
3. Click **ADD NEW VSPHERE ZONE.**
4. In the **vSphere Zone Name** text box, enter **`z-mgmt-a`** and click **NEXT**.
5. Under Datacenter **dc-a**, for the Cluster Selection, select **cluster-mgmt-01a.**
6. Click **FINISH**.
7. Verify that the **z-mgmt-a** is created with an associated cluster **cluster-mgmt-01a** and datacenter **dc-a**.

Task 5: Create VM Storage Policy

You create a VM Storage Policy that is used during creation of the Supervisor cluster.

1. In the vSphere Client, for **vc-mgmt-a.site-a.vcf.lab**, click the **vSphere Menu.**
2. Click **Policies and Profiles**.
3. Click **CREATE** from **VM Storage Policies** section.

   The **Create VM Storage Policy** wizard opens.

4. In the **Name** text box, enter **`mgmt-storage-policy`** and click **NEXT**.
5. On the policy structure wizard, select the following two rules:

a. Select the checkbox for **Enable rules for "vSAN" storage.**

b. Select the checkbox for **Enable Zonal topology for Supervisor.**

c. Click **NEXT.**

6. On the **vSAN** wizard, click **NEXT**.
7. On the **Storage Compatibility** wizard, click **NEXT**.
8. On the **Review and finish** page, click **FINISH**.
9. On the **VM Storage Policies** page, scroll-down to verify that the **mgmt-storage-policy** is created.

Task 6: Activate Supervisor

You activate a Supervisor with VCF Networking with VPC and NSX LB

1. In the vSphere Client, click the **vSphere Menu.**
2. Click **Supervisor Management**.
3. Click **GET STARTED** from the **Supervisor Management** page.

   The **Activate a Supervisor** wizard opens.
4. On the **vCenter Server and Network** page, Select the Networking stack and Load Balancer Type:

   a. In the **Select a networking stack list**, choose the **VCF Networking with VPC** option

   b. Click **NEXT.**
5. On the **Supervisor location** page, enter the Supervisor name and select the vSphere Zone.

   a. Enter **`supervisor-mgmt-a`** in the **Supervisor name** text box.

   b. Don't Toggle **Enable control plane high-availability** button

   c. On the **vSphere Zone** Selection, select **z-mgmt-a**

   d. Click **NEXT**.
6. On the **Storage** page, select the following storage policy:

   a. For all three fields: **Control Plane Storage Policy**, **Ephemeral Disks Storage Policy** and **Image Cache Storage Policy**,

   b. Select **mgmt-storage-policy** from the drop-down menu.

   c. Click **NEXT.**
7. On the **Management Network** page, enter the following configuration details.

a. Click **IP Assignment Mode** drop-down menu.

b. Select **Static** from the drop-down menu.

c. Click on **Select Network(s)** drop-down menu.

d. Select **mgmt-vds01-mgmt-01a** from the **Network** drop-down menu.

e. In the IP Addresses text box, enter **`10.1.1.193-10.1.1.197`**

f. In the Subnet Mask text box, enter **`255.255.255.0`**

g. In the **Gateway** text box, enter **`10.1.1.1`**

h. In the **DNS Server(s)** text, verify that the IP is **`10.1.1.1`**

i. In the **DNS Search Domain(s)** text box, enter **`site-a.vcf.lab`**

j. In the **NTP Server(s)** text box, verify that the IP is **`10.1.1.1`**

k. Click **NEXT** on the Management Network page.

8. On the Workload Network page, Keep **NSX Project** and **VPC Connectivity Profile** to Default. Enter the DNS and NTP Server configuration details.

   a. In the **DNS Server(s)** text-box enter **`10.1.1.1`**

   b. In the **NTP Server(s)** text box, enter **`10.1.1.1`**

   c. Leave all the other values to the defaults and click **NEXT**.

9. On the **Advanced Settings** page, verify that the **Small (CPUs: 4, Memory 16 GB, Storage 48 GB)** is selected.

   a. Select the **Export Configuration** checkbox and click **NEXT**.

10. On the **Ready to complete** page, verify the details and click **FINISH**.

11. Supervisor deployment typically takes around 45 minutes. During this process, the Config state might show **Error** intermittently before the deployment successfully completes. If the process takes significantly longer, check the cluster for any active alerts, including those related to vSAN, and resolve any issues found. Once deployment is complete, ensure that both the **Config Status** and **Host Config Status** are showing as **Running**

Task 7: Verify the status of the Supervisor

You verify the deployment status of the Supervisor.

1. Click on **supervisor-mgmt-a**

   From the Supervisors Summary, verify that the **Configuration Status** is **Running**, **Kubernetes status** is **Ready** and **Node Health** status is **Healthy**

2. Click on **Namespaces** tab.

Verify that the **svc-cci-ns-xxxxx, svc-tkg-xxxxx** and **svc-velero-xxxxx** namespaces **Config Status** is **Running**.

3. Click the **Pods** tab and verify that the Pods status is **Running**.
4. Create the context with VCF CLI for **supervisor-mgmt-a**
   a. Click on **supervisor-mgmt-a**
   b. Click **Summary** tab. Record the **Kubernetes API Server** IP address.
   c. Switch to the **Terminal**.
   d. Create Supervisor admin context with VCF CLI using the supervisor IP address as the endpoint.

      vcf context create sup-mgmt --endpoint=<Kubernetes API Server IP> --insecure-skip-tls-verify --auth-type basic

      Example: **vcf context create sup-mgmt --endpoint=10.1.7.134 --insecure-skip-tls-verify --auth-type basic**

   e. At the Provide username: prompt, enter **administrator@vsphere.local**
   f. At the Provide Password: prompt, enter **VMware123!VMware123!**

      The **Logged in successfully** message appears with the details of the contexts.

   g. View the contents of the **kubeconfig** file

      cat ~/.kube/config

5. List the objects in **supervisor-mgmt-a** cluster
   a. To switch to the Supervisor context, run the following command:

      vcf context use sup-mgmt

      You can safely ignore the following error message during this step:

      ```
      failed to discover plugin sources from the system Harbor registry: the system Harbor registry could not be discovered from the Supervisor cluster.
      ```

      Regional Harbor will be deployed in a subsequent lab module, which will resolve this discovery warning.

   b. List the available contexts. The one marked as Current true indicates the active context. Use the following command:

      vcf context list

   c. List nodes in the Supervisor cluster:

      kubectl get nodes

d. List the services that are exposed using LoadBalancer IP:

```
kubectl get svc -A | grep Load
```

e. Verify that **ServiceClusterReady status** is `True` as Supervisor is deployed using DTGW connectivity profile:

```
kubectl get VPCNetworkConfiguration -n system -oyaml |
grep conditions -A 13
```

Lab 8 Exposing Workloads with Contour Ingress Controller

Objective and Tasks

Install Cert Manager and Contour add-ons, configure ingress routing using Contour HTTPProxy, and deploy an authenticated NGINX workload using Harbor credentials.

1. Connect to VKS Cluster using VKS Cluster Management
2. Install Cert Manager
3. Install Cert Contour
4. Configure Ingress Routing via Contour HTTPProxy
5. Deploy Authenticated NGINX Workload with Harbor Credentials

Task 1: Connect to VKS Cluster using VKS Cluster Management

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - User name: **`admin`**
      - Password: **`VMware123!VMware123!`**
2. Click the **Manage & Govern** tab
3. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
4. Click on VKS cluster **vks-multus01**
5. Click on the **Add-ons** tab

a. Under **Available Add-ons**, verify that **cert-manager** and **contour** Add-ons are available

b. Under **Repositories**, verify that the below default Repository is present.

Name: default-addon-repo-install-3.7.0-20260618

URL: depot.kube-system.svc/vcf/vks-standard-packages/ga/3.7.0-20260618/vks-addons:3.7.0-20260618

Status: Succeeded

Task 2: Installing cert-manager

You install cert-manager Add-on on VKS cluster. Cert Manager is a prerequisite for most standard packages, including Contour.

1. Within the **Add-ons** tab, under **Available Add-ons** section, from **cert-manager** card, click **INSTALL ADD-ON**
2. In **Package name** field, enter **vks-multus01-cert-manager**
3. In **Package version** field, select **cert-manager.kubernetes.vmware.com.1.18.2-vmware.2-vks.2**
4. Under **Package Configuration**, no need to add any data-values yaml in **Overlay YAML** tab for cert-manager.
5. Click **INSTALL ADD-ON** on top right
6. Under **Installed Add-ons** section, a card gets created for the Add-on **vks-multus01-cert-manager**

   Wait for the package to reconcile and status to become green.
7. Verify the resources created in the VKS cluster
   a. Switch to the **Terminal**
   b. Export the downloaded Kubeconfig file

   ```
   export KUBECONFIG=/home/holuser/Downloads/vks-multus01-kubeconfig.yaml
   ```

   c. List the resources in namespace cert-manager

   ```
   kubectl get all -n cert-manager
   ```

Task 3: Installing Contour

You install Contour Add-on on VKS cluster

1. Within the **Add-ons** tab, under **Available Add-ons** section, from Contour card, click **INSTALL ADD-ON**
2. In **Package name** field, enter **vks-multus01-contour**
3. In **Package version** field, select **contour.kubernetes.vmware.com.1.33.4-vmware.1-vks.1**
4. Under **Package Configuration**, copy the content of below data-values yaml and paste it in **Overlay YAML** tab

   `/home/holuser/Lab-files/package-yaml/contour-data-values.yaml`
5. Click **INSTALL ADD-ON** on top right
6. Under **Installed Add-ons** section, a card gets created for the Add-on **vks-multus01-contour**

   Wait for the package to reconcile and status to become green.
7. Verify the resources created in the VKS cluster
   a. Switch to the **Terminal**
   b. Export the downloaded Kubeconfig file

      `export KUBECONFIG=/home/holuser/Downloads/vks-multus01-kubeconfig.yaml`
   c. List the resources in namespace tanzu-system-ingress

      `kubectl get pods,svc -n tanzu-system-ingress`

Task 4: Configure Ingress Routing via Contour HTTPProxy

1. Create a namespace:

   `kubectl create namespace nginx`
2. Label the namespace

   `kubectl label ns nginx pod-security.kubernetes.io/enforce=privileged`
3. Review nginx-httpproxy.yaml at `~/Lab-files/nginx/nginx-httpproxy.yaml`
   a. Retrieve the Ingress LoadBalancer IP:

      `kubectl get svc -n tanzu-system-ingress`
   b. Update the **virtualhost** field in the second manifest with this IP address.
   c. Apply the Manifest

```
kubectl apply -f ~/Lab-files/nginx/nginx-httpproxy.yaml
```

Task 5: Deploy Authenticated NGINX Workload with Harbor Credentials

1. Create image pull secret for Harbor registry

   ```
   kubectl create secret docker-registry harbor-pullsecret \
     --namespace=nginx \
     --docker-server=10.100.6.4 \
     --docker-username=nginx_admin \
     --docker-password='VMware123!VMware123!'
   ```

2. Review application manifest nginx-app.yaml at `~/Lab-files/nginx/nginx-app.yaml`

3. Apply the manifest file

   ```
   kubectl apply -f ~/Lab-files/nginx/nginx-app.yaml
   ```

4. Verify the pods status

   ```
   kubectl get pods -n nginx
   ```

5. verify the the HTTPProxy status

   ```
   kubectl get httpproxy -n nginx
   ```

6. Open Google Chrome and access the application via Envoy IP **EXTERNAL-IP**

7. Verify that the "**Welcome to nginx!**" landing page displays successfully.

Lab 9 Deploying Containers Using VCF Automation

Objective and Tasks

You deploy and manage containers using the container service in VCF Automation.

1. Deploy a Container using the Container Service
2. Verify the Container Details
3. Scale up the Container
4. Verify the Pod Details in vCenter

Task 1: Deploy a Container using the Container Service

You Deploy a container using VCF Automation container service.

1. Log in to the VCF Automation acme-prod Organization Portal.
    a. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
    b. In the **Organization name**, enter **acme-org**
    c. Log in to the **acme-org** Organization portal using Org admin credentials
        - Log in with local account
        - **Username**: **`admin`**
        - **Password**: **`VMware123!VMware123!`**
2. Click the **Build & Deploy** tab.
    a. Ensure **acme-prod-xxxx** namespace is selected under the Services section in the left pane.
3. Navigate to **Services > Container** from the left pane.

4. Click **CREATE NEW INSTANCE**. Use the settings below:

Container Instance Name	**webserver1**
Zone	**z-wld-a**
Registry Authentication	**Leave unchecked**
Primary Container Image	**bitnami/nginx**
Primary Container Resources	**Required CPU: 250 millicores** **Required Memory: 256 MB** **Replicas: 2**

   a. Click **NEXT.**

5. Under **Advanced settings (optional)**, expand **Load Balancer**

   a. Click **+ ATTACH LOAD BALANCER**.

   b. In the Load Balancer **Name** field, enter **webserver1-lb**

   c. In **Create new port** section, use the settings below:

Name	**http**
Protocol	**TCP**
Port	**80**
Target	**8080**
Attach Primary Container	**Select the checkbox**

   Click **ADD**

   d. Click **SAVE**

   e. Click **NEXT**

6. Click **CREATE CONTAINER INSTANCE**

Task 2: Verify the Container Details

You verify the container details in the VCF Automation console.

1. On the Container Service page, click the **webserver1** hyperlink.

You are taken to the webserver1 page.

a. Check the Pods status. Phase should be **Running**

b. Under **Load Balancers**, **External IP** should have been assigned

Open another tab on **Firefox** browser and run the command - **`http://<External-Ip>`**

You should see **Welcome to nginx!** page

Task 3: Scale up the Container

You scale up the container from the VCF Automation console.

1. On the Container Service page, click the **webserver1** hyperlink.
2. Click the **SCALE** option on the top.
3. Increase the replica count to **3**.
   a. Click **SAVE**.

   You may need to refresh the browser page to see the changes.
4. Verify that the instance status is now in **Not Ready** state. New pod is added and is in **Pending** state.
   a. Wait for the new pod to complete the deployment.
5. Verify that there are three replicas in the running state and the instance is in **Ready** state.

Task 4: Verify the Pod Details in vCenter

You login to vCenter and validate the vSphere pod details.

1. On the browser menu, click on **Region A > vc-wld01-a**
   a. The vCenter Login page is displayed.
2. Login with the credentials below

   Username: **`administrator@wld.sso`**

   Password: **`VMware123!VMware123!`**
3. Click the menu icon on the top left hand corner.
   a. Click **Inventory**.
4. Under **Namespaces**, expand the **acme-prod-xxxxx** namespace.
5. Verify that three instances of the **webserver1** pods are running as expected.

6. Click the menu icon on the top left hand corner.
    a. Select **Supervisor Management.**
7. Click **acme-prod-xxxxx** under the **Namespaces** section.
8. Go to the **Resources** tab.
    a. Under the container tile click on **GO TO SERVICE.**
9. Click the two arrows next to **webserver1** to expand it.
    a. Verify the details and note the three pods running under the pods section as expected.

Lab 10 Deploying MySQL DB VM in Tenant Organization

Objective and Tasks

You deploy a VM from the Organization Portal using Organization user.

1. Log In to the VCF Automation Organization Portal
2. Deploy MySQL DB Virtual Machine

Task 1: Log In to the VCF Automation Organization Portal

You log in to the VCF Automation acme-org Organization Portal.

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. In the **Organization name**, enter **acme-org**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - **Username**: **`admin`**
      - **Password**: **`VMware123!VMware123!`**

Task 2: Deploy MySQL DB Virtual Machine

You deploy the Virtual Machine from the Organization Portal.

1. In the Organization Portal, click **Build & Deploy.**
2. Click **Virtual Machine** from Services menu in the left.
3. Click **+ CREATE VM**
4. In the **New Virtual Machine** Wizard, select **Deploy from OVF** and click **NEXT.**

5. In the **VM Name** text box type **`oc-mysql`**
6. From the **Infrastructure Policy** drop-down menu, keep it None
7. From the **Zone** drop-down menu, select **z-wld-a**
8. From the **VM Group** drop-down menu, keep it None
9. Select **ubuntu24-mysql,** under **VM Image.**
    a. Scroll-down to select VM Class **best-effort-small**, Storage Class **wld-storage-policy** and Power State **Powered On**
10. Click **NEXT**.
11. Complete the Load Balancer configuration.
    a. In the Load Balancer section, click **ADD** drop-down menu.
    b. Click **New**
    c. On the **Create VM Load Balancer** window, Edit the **Name** to **vm-lb-mysql-db**

       In **Create new port** section, create a first port using the below details.

Option	Value
Name	ssh
Port	22
Protocol	TCP
Target	22

    d. Click **ADD**.
    e. In **Create new port** section, create the second port using the below details.

Option	Value
Name	mysql
Port	3306
Protocol	TCP
Target	3306

    f. Click **ADD**.

g. Click **SAVE**.

12. Scroll down to the Guest Customization and select **CLOUD-INIT**
13. On **Configure via** page, click **RAW CONFIGURATION**
    a. On the **RAW CONFIGURATION** page, copy and paste the configuration from the **oc-mysql-cloud-config.yaml** file.
        i. From the taskbar click on **Files** folder.
        ii. Navigate to **Home > Lab-files > opencart** folder.
        iii. Double-click to open **oc-mysql-cloud-config.yaml** file
        iv. Press **CTRL + A** to select all the content from **oc-mysql-cloud-config.yaml** file.
        v. Press **CTRL + C** to copy the content.
        vi. Return to the **RAW CONFIGURATION** page and paste the data in the text box.
        vii. Click **NEXT.**
14. In the **Global Network Settings**, leave the fields blank and click **NEXT**
15. On the Review and Confirm page, click **DEPLOY VM.**

    The oc-mysql virtual machine deployment will take few mins to complete.
16. Verify that the mySQL VM is in a **Ready** and **Powered ON** state.
17. Verify the External IP details.

    In the left navigation pane under **Services** > **acme-prod-xxxx,** click **Network**. Select the **VM Load Balancers** tab, then expand **vm-lb-mysql-db.**

    Record the **External IP** assigned to **vm-lb-mysql-db.** This will be used during OpenCart application deployment.

Lab 11 Deploying a VKS Cluster in Tenant Organization

Objective and Tasks

You deploy VKS Cluster from the VCF Automation Organization portal.

1. Log in to the VCF Automation Organization Portal
2. Deploy VKS Cluster
3. Verify the VKS Cluster Configuration

Task 1: Log In to the VCF Automation Organization Portal

You log in to the VCF Automation Organization Portal.

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - **Username**: **`admin`**
      - **Password**: **`VMware123!VMware123!`**

Task 2: Deploy VKS Cluster

1. In the Organization portal, click on **Build & Deploy** tab.
2. Click **Kubernetes** under Services.
3. Click **+ CREATE.**

   The **New Kubernetes Cluster** wizard opens.

4. On the **Configuration Type**, select the **Custom Configuration** and click **NEXT**.
5. On the **General Settings**, enter the details below.

Parameter	Value
Cluster Name	**vks-mt-app01**
Cluster Class	**builtin-generic-v3.6.0**
Kubernetes	**v1.35.5---vmware.1-vkr.1**
VM Class	**best-effort-small**
Storage Class	**wld-storage-policy**

6. Expand **Network** and under **Pod Network**, enter the details below:

Parameter	Value
POD CIDR	**192.168.156.0/20**
Services CIDR	**12.96.0.0/12**
Service Domain	**cluster.local**

7. Scroll down to the end of the **General Settings** page, and click **NEXT**.
8. Under Control plane, Click **Replicas** drop-down menu and select **3**.
9. From the **OS Image** drop-down menu, verify that the **Photon 5 - Kubernetes Service Content Library** is selected.
10. Click **NEXT** to proceed to the **Node pools** section.
11. Click **vertical dotted line** before **vks-mt-app01-np-xxxx** and then click **Edit** to edit the Nodepool configuration.

    The Configuration wizard opens.

Option	Value
Name	Modify the name to **vks-mt-app01-np-01**
Class	**node-pool**
Zone	**z-wld-a**

Replicas	**3**
OS Image	**Photon 5 - Kubernetes Service Content Library**

a. Click **NEXT**.

b. Click **FINISH**.

12. Click **NEXT**.
13. Click **FINISH**.

    You monitor the VM deployment in the Workload vSphere Client during VKS Cluster deployment.

    VKS Cluster deployment typically takes around 15 minutes.

Task 3: Verify the VKS Cluster Configuration

1. On **Kubernetes** Page under **Services**, click **vks-mt-app01**
2. Click **DOWNLOAD KUBECONFIG FILE.** This Kubeconfig is used to connect to the VKS Cluster
3. Click **Save**
4. Verify the cluster Details from the **Summary** tab.

Options	Value
Status	**Available**
Kubernetes Release	**v1.35.5--vmware.1-vkr.1**
Certificate Rotation	**Enabled; 90 Days**
CNI	**Antrea**
Control Plane Replicas	**3**
Node Pool Replicas	**3**

5. Click **Network** under **Services**.
6. Click on **VM Load Balancers** tab and expand **vks-mt-app01**
7. Review the Service Details.

Options	Value
Type	Load Balancer
External IP	**10.100.6.x** (ex: 10.100.6.4)
Ports	6443/TCP

8. Switch to the **Terminal**
   a. Export the downloaded Kubeconfig file

      ```
      export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml
      ```

   b. List the nodes of the VKS Cluster

      ```
      kubectl get nodes
      ```

   c. Close the **Terminal**

Lab 12 Connecting to VKS Cluster using SSH

Objective and Tasks

You connect to the VKS cluster using SSH

1. Fetch VKS cluster SSH password
2. Create a Jumpserver to connect to the VKS cluster

Task 1: Fetch VKS cluster SSH password and Node IP address

1. Open a new **Terminal** from taskbar
2. Switch to the Supervisor context

   vcf context use sup-wld

   At **Password** prompt, enter **VMware123!VMware123!**

   Get the namespace of where VKS Cluster is deployed

   kubectl get ns | grep acme-prod

   List the Cluster

   vcf cluster list -n acme-prod-xxxx
3. Retrieve the VKS cluster SSH password by running the below command:

   kubectl get secret -n acme-prod-xxxxx vks-mt-app01-ssh-password -o jsonpath={.data.ssh-passwordkey} | base64 -d
4. Get IP addresses of the VKS cluster nodes

   kubectl get vspheremachines -n acme-prod-xxxx | grep vks-mt-app01

Note down the **VKS cluster SSH password** and the **vspheremachines** (Nodes) IP addresses

As VKS cluster node IPs are assigned from the **VPC Private CIDR**, they are not reachable from outside the Project.
In the next Task, we will create a Jumpserver VM in the same namespace so that the VKS cluster nodes are reachable from inside the Jumpserver VM.
We will assign a **LoadBalancer** IP to the Jumpserver VM to enable external access.

Task 2: Create a Jumpserver to connect to the VKS cluster

You create a Jumpserver virtual machine in acme-org

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation**
   a. Under Organization name, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Login using the Local user
      - **Username**: **`admin`**
      - **Password**: **`VMware123!VMware123!`**
2. In the Organization portal, click **Build & Deploy.**
3. Under **Services** on left panel, click **Virtual Machine**
4. Under Virtual Machines tab, click **+ CREATE VM**
5. In the **New Virtual Machine** wizard, select **Deploy from OVF** and click **NEXT.**
6. In the **VM Name** text box, enter **`jumpserver`**
7. From the **Zone** drop-down menu, select **z-wld-a**
8. Select **ubuntu24-mysql** under VM Image.
   a. Scroll-down to select **best-effort-small**, under VM Class and Storage Class **wld-storage-policy**

Click **Next**

9. Complete the Load Balancer configuration.
   a. In the Load Balancer section, click **ADD** drop-down menu.
   b. Click **New**
   c. On the Create VM Load Balancer window, Edit the **Name** to **jumpserver-lb**

      In **Create new port** section, create a first port using the below details.

Option	Value
Name	ssh
Port	22
Protocol	TCP
Target	22

d. Click **ADD**.

e. Click **SAVE**.

10. Scroll down to the **Guest Customization** and verify that **cloud-init** is selected.
11. Click **CREATE NEW USER**.

    The **Create New User** window opens.

Option	Value
Username	**ubuntu-user**
Password	**VMware123!VMware123!**
Confirm Password	**VMware123!VMware123!**
Default Sudo	**Enable**

a. Click **SAVE**.

b. Click the **SSH Password Authentication** drop-down menu and select **Enable**.

c. Click **NEXT**.

12. Under **Global Network Settings**, click **NEXT**.
13. Click **DEPLOY VM**.

    The virtual machine deployment will take few minutes.
14. Click the virtual machine Name (jumpserver) in the **Name** column to verify the VM Details.
15. In the left navigation pane under **Services** > **acme-prod-xxxx,** click **Network**. Select the **VM Load Balancers** tab, then expand **jumpserver-lb.**

    Verify that the **External IP** is assigned.
16. SSH to the Jumpserver VM using **External IP**

ssh ubuntu-user@<External-IP>

Enter password **VMware123!VMware123!**

17. From inside the Jumpserver VM, SSH to the VKS cluster

    ssh vmware-system-user@<Node-IP>

    Enter the Node-IP from the previous task

    Enter password from the previous task

    Make sure to copy the entire content before **holuser** as password. If the console access is slow, paste the password and wait a few seconds before pressing Enter.

18. Close the **Terminal**

Lab 13 Deploying OpenCart Application

Objective and Tasks

You deploy OpenCart application using kubectl

1. Deploy OpenCart Application

Task 1: Deploy OpenCart Application

1. Click the **Terminal** icon from the task bar to open the command line.

   OpenCart Application YAML manifest files are present at - `/home/holuser/Lab-files/opencart`

2. Export the downloaded Kubeconfig file

   `export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml`

   a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

      `kubectl get nodes`

      Above command should list **vks-mt-app01** nodes

3. Create the namespace

   `kubectl create namespace opencart`

4. Label the namespace

   `kubectl label ns opencart pod-security.kubernetes.io/enforce=privileged`

5. Deploy a LoadBalancer service in the opencart namespace to expose the application externally.

   cd /home/holuser/Lab-files/opencart

   kubectl apply -f opencart-lb.yaml

   Note down the **External-IP** assigned to the LoadBalancer service from the output of below command. E.g. 10.100.6.6

   kubectl get svc -n opencart

6. Modify the OpenCart YAML manifest file **opencart.yaml** to match your environment settings.

   vi /home/holuser/Lab-files/opencart/**opencart.yaml**

   Update the IP address below to match the MySQL database VM's Load Balancer IP. In the Organization Portal, go to **Build & Deploy > Network > Services > vm-lb-mysql-db > External IP**

```
        - name: OPENCART_DATABASE_HOST
          value: '10.100.6.8'
```

   Update the IP address below to match the OpenCart Loadbalancer IP. Run the command to get the external IP assigned to opencart load balancer **kubectl get svc -n opencart | grep Load**

```
        - name: OPENCART_HOST
          value: '10.100.6.6'

      livenessProbe:
        failureThreshold: 6
        httpGet:
          httpHeaders:
            - name: Host
              value: '10.100.6.6'

      readinessProbe:
        failureThreshold: 6
        httpGet:
          httpHeaders:
            - name: Host
              value: '10.100.6.6'
```

Verify that the image path references the Regional Harbor FQDN. Image pull operations will fail if the Regional Harbor IP address is used instead of the FQDN.

```
          image: depot.kube-
system.svc/opencart/opencart:4.0.1-1-debian-11-r66
```

7. Deploy the OpenCart Application

   `kubectl apply -f opencart.yaml`

   Below message appears:
   persistentvolumeclaim/open-cart-pvc created
   secret/open-cart-oc-password created
   secret/my-open-externaldb created
   deployment.apps/my-open-cart-app created

8. Verify the resources created for the OpenCart Application

   `kubectl get all -n opencart`

   `kubectl get pvc -n opencart`

9. Open **Firefox** from the taskbar.

   Open a **new tab** in **Firefox**.

   a. In the address bar type the **`http://EXTERNAL-IP`** and hit enter.

      e.g, **`http://10.100.6.6`**

      The **OpenCart** page appears.

10. Close the **Terminal** and **Firefox** tab

Lab 14 Deploying Applications using Helm

Objective and Tasks

You deploy Nginx application using Helm

1. Upload Images and Helm Charts to Harbor
2. Deploy Nginx using Helm

Task 1: Upload Images and Helm Chart to Harbor.

You create a project in Harbor and upload the Nginx Images and Helm Charts to the project

You can skip this task, as both images and Helm charts have already been uploaded to Harbor.

1. Enter **`https://<Harbor IP>`** in the address bar and press Enter.
2. Log in to Harbor:
   - **Username**: **`admin`**
   - **Password**: **`VMware123!VMware123!`**
3. Under **Project**, click **+ NEW PROJECT**.
4. On the **New Project** window, enter **`nginx`** in the Project Name text box
5. Select the **Public** checkbox for Access Level and click **OK**
6. Click **Users** under **Administration**.
7. Click **+ NEW USER**
8. On the **New User** enter the following configuration.

Option	Value

Username	**nginx_admin**
Email	**nginx_admin@vcf.lab**
First and Last name	**nginx admin**
Passsword	**VMware123!VMware123!**
Confirm Password	**VMware123!VMware123!**

a. Click **OK**.

9. Under **Project**, click **nginx**
10. Click **Members** tab and Click **+ USER.**
11. On the new Member, complete the following details:

Option	Value
Name	**nginx_admin**
Role	**Maintainer**

a. Click **OK**.

12. Login to Docker

    Click **Terminal** from the taskbar.

    a. Change the directory:

    cd Lab-files/nginx/

    b. Retrieve the Harbor SSL certificate

    openssl s_client -showcerts -connect <Harbor IP>:443 < /dev/null | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > ca.crt

    Example - **openssl s_client -showcerts -connect 10.100.6.4:443 < /dev/null | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > ca.crt**

    c. Create the directory that will be used by Docker to trust the Harbor registry

    sudo mkdir -p /etc/docker/certs.d/<Harbor IP>/

    Example - **sudo mkdir -p /etc/docker/certs.d/10.100.6.4/**

At the [sudo] password for holuser: prompt, enter **VMware123!VMware123!**

d. Copy Certificate.

sudo cp ca.crt /etc/docker/certs.d/<harbor IP>/ca.crt

Example - **sudo cp ca.crt /etc/docker/certs.d/10.100.6.4/ca.crt**

e. Log in to docker.

Logout any existing user session

sudo docker logout 10.100.6.4

Enter Password for **[sudo] password for holuser**: **VMware123!VMware123!**

Perform docker login using nginx_admin user

sudo docker login 10.100.6.4

On the Username: prompt, type **nginx_admin** and press Enter.

On the Password: prompt, type **VMware123!VMware123!** and press Enter.

The Login Succeeded message appears.

13. Load the Image

List the Nginx Image

ls -ltr

Load the image **nginx_image.tar**

sudo docker image load -i /home/holuser/Lab-files/nginx/nginx_image.tar

Loaded image: bitnami/nginx:latest message appears

14. Tag the image with Harbor registry FQDN and nginx project name

sudo docker tag bitnami/nginx:latest 10.100.6.4/nginx/nginx:latest

15. Push the tagged image to the nginx repository in Harbor.

sudo docker push 10.100.6.4/nginx/nginx:latest

16. Upload Helm Chart to Harbor

helm registry login 10.100.6.4

On the Username: prompt, type **nginx_admin** and press Enter.

On the Password: prompt, type **VMware123!VMware123!** and press Enter.

```
helm push nginx-22.1.1.tgz oci://10.100.6.4/nginx
```

17. Verify Images and Helm Chart on Harbor

    Log into Harbor UI, navigate to the **nginx** project

    Click **Repositories** tab and then click **nginx/nginx**

    Verify that the pushed image and helm chart appears in the project repository. Each artifact type is denoted by an icon: a Docker logo for images and a Helm logo for charts.

Task 2: Deploy Nginx using Helm

You deploy Nginx application in vks-mt-app01 using helm install

1. Open a new **Terminal** from the taskbar

   Export the **kubeconfig** file to connect to **vks-mt-app01**

   ```
   export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml
   ```

   a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

      ```
      kubectl get nodes
      ```

      Above command should list **vks-mt-app01** nodes

2. Create a namespace for deploying nginx

   ```
   kubectl create namespace nginx
   ```

   namespace/nginx created mesage appears

3. Remove PSA restrictions on the namespace

   ```
   kubectl label ns nginx pod-security.kubernetes.io/enforce=privileged
   ```

   namespace/nginx labeled message appears

4. Create a Harbor image pull secret to access images.

   ```
   kubectl create secret docker-registry harbor-pullsecret --docker-server=<harbor IP> --docker-username=nginx_admin --docker-password='VMware123!VMware123!' --docker-email=nginx_admin@vcf.lab --namespace=nginx
   ```

Example - **kubectl create secret docker-registry harbor-pullsecret --docker-server=10.100.6.4 --docker-username=nginx_admin --docker-password='VMware123!VMware123!' --docker-email=nginx_admin@vcf.lab --namespace=nginx**

secret/harbor-pullsecret created message appears

5. Create an nginx-values.yaml file to override the chart's default values, specifying Harbor details and the image pull secret.

   Review the file **/home/holuser/Lab-files/nginx/nginx-values.yaml**

6. Authenticate local Helm client against the OCI registry at 10.100.6.4 so you can push or pull Helm charts stored as OCI artifacts.

   helm registry login <Harbor IP> -u nginx_admin -p 'VMware123!VMware123!' --ca-file /etc/docker/certs.d/<Harbor IP/ca.crt

   Example - **helm registry login 10.100.6.4 -u nginx_admin -p 'VMware123!VMware123!' --ca-file /etc/docker/certs.d/10.100.6.4/ca.crt**

7. Deploy nginx using the Helm install

   helm install nginx oci://<Harbor IP>/nginx/nginx --version 22.1.1 -n nginx -f nginx-values.yaml --set global.security.allowInsecureImages=true

   Example - **helm install nginx oci://10.100.6.4/nginx/nginx --version 22.1.1 -n nginx -f nginx-values.yaml --set global.security.allowInsecureImages=true**

   Retry if you see any error or timeout

8. Verify that the status of helm install

   helm list -n nginx

9. Verify the status of the kubernetes objects

   kubectl get all -n nginx

   Verify that pods are in **Running** state and **EXTERNAL-IP** has been assigned to the service nginx

10. Access the Nginx page

    On the **Firefox**, enter http://<**EXTERNAL-IP**> in the address bar and press Enter.

11. Close the **Terminal** and **Firefox** tab

Lab 15 Upgrading and Rolling Back Applications using Helm

Objective and Tasks

You upgrade Nginx application using Helm

1. Upload New Helm Chart to Harbor
2. Upgrade Nginx using Helm
3. Rollback to a Previous Revision

Task 1: Upload New Helm Chart to Harbor

You upload the new Helm Chart to nginx project in Harbor

You can skip this task, as both images and Helm charts have already been uploaded to Harbor.

1. Open a new **Terminal** from the taskbar.
   a. Change the directory:

      `cd Lab-files/nginx/`

2. Upload Helm Chart to Harbor

   `helm registry login 10.100.6.4`

   On the Username: prompt, type **`nginx_admin`** and press Enter.

   On the Password: prompt, type **`VMware123!VMware123!`** and press Enter.

   `helm push nginx-22.2.1.tgz oci://10.100.6.4/nginx`

3. Verify Helm Chart on Harbor

   On the **Firefox** Browser, enter in the address bar and press Enter. Log in to Harbor

Log in to Harbor

Username: **admin**

Password: **VMware123!VMware123!**

a. Under **Project**, click **nginx**

Click **Repositories** tab and then click **nginx/nginx**

b. Verify that the pushed new helm chart appears in the project repository

Task 2: Upgrade Nginx using Helm

You upgrade Nginx by performing Helm upgrade

1. Switch to **Terminal**

Export the **Kubeconfig** file to connect to **vks-mt-app01**

```
export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml
```

a. Verify that the correct **kubeconfig** was exported by listing the nodes of the VKS Cluster

```
kubectl get nodes
```

Above command should list **vks-mt-app01** nodes

2. Upgrade nginx using the Helm upgrade

```
helm upgrade nginx oci://<Harbor IP>/nginx/nginx --version 22.2.1 -n nginx -f nginx-values.yaml --set global.security.allowInsecureImages=true
```

Example - **helm upgrade nginx oci://10.100.6.4/nginx/nginx --version 22.2.1 -n nginx -f nginx-values.yaml --set global.security.allowInsecureImages=true**

3. Verify that the status of helm upgrade

helm list -n nginx

Verify the below details

Option	Value
REVISION	2
STATUS	deployed

CHART	nginx-22.2.1

4. Verify the kubernetes objects

   kubectl get all -n nginx

   Verify that new replicaset has been created and old pods have been replaced by new pods

Task 3: Rollback to a Previous Revision

You rollback Nginx by performing Helm rollback

1. Check the deployment history to view active and past revisions of the Nginx release.

   helm history nginx -n nginx

2. Roll back the deployment to Revision 1 (the initial installation)

   helm rollback nginx 1 -n nginx

3. Remove the Nginx release and all associated resources from the namespace.

   helm uninstall nginx -n nginx

4. Close the **Terminal**

Lab 16 Create multi-nic nodes on a VKS Cluster

Objective and Tasks

You perform the below tasks:

1. Create customised SubnetSet for Primary Network
2. Create Parent Subnet
3. Create Child Overlay Subnet
4. Create SubnetConnectionBindingMap for the Child Overlay Subnet and the Parent Subnet
5. Create a Distributed VLAN connection from VCF Automation Provider Management Portal
6. Create VLAN extension subnet from VCF Automation Org Portal
7. Create SubnetConnectionBindingMap for the Child VLAN extension Subnet and the Parent Subnet
8. Reserve an IP range for the pod from the Child Subnets
9. Enable featureGates in Antrea configuration
10. Create a multi-nic cluster
11. Verify the interfaces on VKS cluster nodes

Task 1: Create customised SubnetSet for Primary Network

You create a SubnetSet with customised settings on workload domain Supervisor.

1. Open a new **Terminal** from the taskbar.
2. List VCF Contexts.

```
vcf context list
```

3. Switch to Supervisor admin context

   ```
   vcf context use sup-wld
   ```

   If prompted for **Provide Password**, type **`VMware123!VMware123!`** and press Enter.

4. Create the **SubnetSet** with **accessMode Public**

   Retrieve the vSphere Namespace name provisioned by VCF Automation

   ```
   kubectl get ns | grep acme-prod
   ```

   Navigate to the vks-files folder.

   ```
   cd /home/holuser/Lab-files/vks-3.6
   ```

   Review **primary-subnetset.yaml** and update the namespace **acme-prod-xxxxx** as per your environment if needed.

   a. Apply the **primary-subnetset.yaml** manifest file to create the **Subnetset**

      ```
      kubectl apply -f primary-subnetset.yaml
      ```

   b. Verify that **subnetset-public** is created in **acme-prod-xxxxx** namespace

      ```
      kubectl get subnetset -n acme-prod-xxxxx
      ```

Task 2: Create Parent Subnet

You create a subnet for the node vNIC as the parent and disable static IP allocation to avoid IP exhaustion

1. Within the **Lab-files/vks-3.6/** directory, review **parent-subnet.yaml** and update the namespace **acme-prod-xxxxx** as per your environment if needed.
2. Apply the **parent-subnet.yaml** manifest file to create the **Subnet**

   ```
   kubectl apply -f parent-subnet.yaml
   ```

3. Verify that **secondary-node-net** has been created in **acme-prod-xxxxx** namespace

   ```
   kubectl get subnet -n acme-prod-xxxxx
   ```

Task 3: Create Child Overlay Subnet

You create a child overlay subnet in your vSphere Namespace

1. Within the **Lab-files/vks-3.6/** directory, review **child-overlay-subnet.yaml** and update the namespace **acme-prod-xxxxx** as per your environment.
2. Apply the **child-overlay-subnet.yaml** manifest file to create the **Subnet**

```
kubectl apply -f child-overlay-subnet.yaml
```

3. Verify that **secondary-pod-net** has been created in **acme-prod-xxxxx** namespace

```
kubectl get subnet -n acme-prod-xxxxx
```

Task 4: Create Child Overlay Subnet to Parent Subnet Binding

You bind the child subnet **secondary-pod-net** to the parent subnet **secondary-node-net** by using the SubnetConnectionBindingMap

1. Within the **Lab-files/vks-3.6/** directory, review **overlay-subnet-binding.yaml** and update it with your **acme-prod-xxxxx** namespace
2. Apply the **overlay-subnet-binding.yaml** manifest file to create the **SubnetConnectionBindingMap**

```
kubectl apply -f overlay-subnet-binding.yaml
```

3. Verify that **secondary-overlay-subnet-binding** has been created in **acme-prod-xxxxx** namespace

```
kubectl get subnetconnectionbindingmap -n acme-prod-xxxxx
```

Task 5: Create a Distributed VLAN connection from VCF Automation Provider Management Portal

You create a Distributed VLAN connection in the Provider Management Portal and share it with the organization.

1. Log in to the VCF Automation Provider Management Portal.
   a. From your desktop, open **Firefox** browser. From the bookmarks toolbar, click **Region A** > **VCF Automation**
   b. If the `Your connection is not private message` appears, click **Advanced** and click **Proceed to auto-a.site-a.vcf.lab (unsafe).**
   c. Log in to the VCF Automation Provider Management Portal using the following saved credentials. If there is a need to specify the Organization in the portal, it is **System**.
      - **Username**: **admin**
      - **Password**: **VMware123!VMware123!**
2. Under **Networking**, click on **External Connections**
3. Navigate to **Distributed VLAN Connections** tab and click **New**
4. Enter the Name **vlan-extension-connection**

5. Select the Region **us-east** and click **NEXT**
6. Enter the below details for the **Connection Details**

Parameter	Value
VLAN ID	**27**
Gateway CIDR IP Address	**10.1.10.1/25**
Create a Distributed VLAN Connection dedicated to a Subnet	**On**

   Click **NEXT**
7. In **Review and Complete** section, review the inputs and click **CREATE**
8. Assign the VLAN connection you just created as the dedicated VLAN connection for the Organization
   a. Under **Infrastructure**, Navigate to **Organizations** and click on **acme-org**
   b. Navigate to **Networking** > **Connections**
   c. Click on **NEW DEDICATED VLAN CONNECTION**
   d. Select **vlan-extension-connection** and click **ADD**
9. Navigate to **External Connections** under **Networking**
   a. Click on **Distributed VLAN Connections**
   b. Verify that Distributed Connection **vlan-extension-connection status** is **Normal** and **Used by** is **Dedicated > Acme-Org**
10. Log out from VCF Automation Provider Management Portal

Task 6: Create VLAN extension subnet from VCF Automation Org Portal

You create a VLAN extension subnet from the VCF Automation Organization Portal

1. Launch acme-org Organization Portal. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. In the **Organization name**, enter **acme-org**
   b. Log in to the **acme-org** Organization portal using Org admin credentials
      - Log in with local account
      - **Username**: **admin**

- Password: **VMware123!VMware123!**

2. Navigate to **Manage & Govern** tab
3. Under **Networking**, click on **Subnets**
4. Click **NEW SUBNET**
5. Create a Subnet with the below details

Parameter	Value
Region	**us-east**
VPC	**default-us-east**
Name	**default-us-east:vlan-extension-subnet**
VLAN Connection	**Yes**
Distributed External Connection	**vlan-extension-connection**

   Click **CREATE**

6. Add the VLAN extension subnet to that Organization Namespace.
   a. From **Manage & Govern** tab, navigate to **Namespaces**
   b. Click 3 dots on the namespace **acme-prod-xxxxx** and click **Edit**
   c. Click **Edit** on the **Subnets**
   d. Select the Shared subnet **default-us-east:vlan-extension-subnet**
   e. Click **SAVE**

Task 7: Create Child VLAN Extension Subnet to Parent Subnet Binding

You bind the child subnet **vlan-extension-subnet** to the parent subnet **secondary-node-net** by using the SubnetConnectionBindingMap

1. Switch back to **Terminal**
2. Within the **Lab-files/vks-3.6/** directory, review **vlan-subnet-binding.yaml** and update it with your **acme-prod-xxxxx** namespace name.
3. Apply the **vlan-subnet-binding.yaml** manifest file to create the **SubnetConnectionBindingMap**

```
kubectl apply -f vlan-subnet-binding.yaml
```

4. Verify that **secondary-vlan-subnet-binding** has been created in **acme-prod-xxxxx** namespace

```
kubectl get subnetconnectionbindingmap -n acme-prod-xxxxx
```

Task 8: Reserve an IP range for the pod from the Child Subnets

You reserve an IP range for the pod from the child subnets for the pod to avoid conflicts between the NSX and the Antrea IPAM

1. Within the **Lab-files/vks-3.6/** directory, review **pod-subnet-reservation.yaml** and update the namespace **acme-prod-xxxxx** as per your environment.
2. Apply the pod-subnet-reservation.yaml manifest file to create the **SubnetIPReservation**

```
kubectl apply -f pod-subnet-reservation.yaml
```

3. Verify that **reserved-ip-on-secondary-pod-net** and **reserved-ip-on-vlan-extension-subnet** have been created in **acme-prod-xxxxx** namespace

```
kubectl get subnetipreservation -n acme-prod-xxxxx
```

Task 9: Enable featureGates in Antrea configuration

You set the Antrea configuration with the specified networks and activate AntreaIPAM and SecondaryNetwork

1. Within the **Lab-files/vks-3.6/** directory, review **antrea-config.yaml** and update the namespace **acme-prod-xxxxx** as per your environment.
2. Apply the **antrea-config.yaml** manifest file to create the **AddonConfig**

```
kubectl apply -f antrea-config.yaml
```

addonconfig.addons.kubernetes.vmware.com/vks-multinic01-antrea created message appears

3. Verify that **vks-multinic01-antrea** has been created in **acme-prod-xxxxx** namespace

```
kubectl get addonconfig -n acme-prod-xxxxx
```

Task 10: Create a multi-nic cluster

You create a multi-nic VKS cluster referencing the pre-created SubnetSet for the primary interface and the parent subnet for the secondary interface

1. Within the **Lab-files/vks-3.6/** directory, review **vks-multinic01.yaml** and update the namespace **acme-prod-xxxxx** as per your environment if needed.

Parameter	Value
namespace	**acme-prod-xxxx**
version	**v1.35.5---vmware.1-vkr.1**
storageClass	**wld-storage-policy**

2. Apply the **vks-multinic01.yaml** manifest file to create the **VKS cluster**

   kubectl apply -f vks-multinic01.yaml

3. Verify that **vks-multinic01** has been created in **acme-prod-xxxxx** namespace

   kubectl get cluster -n acme-prod-xxxxx

   Wait for the **AVAILABLE** status to become **True**

   Verify the vspheremachine status

   kubectl get vspheremachine -n acme-prod-xxxxx -o wide

   Record the VKS Cluster worker node IP address.

4. Verify the interfaces on the worker node. It should have Primary and Secondary interfaces.

   Use the **vspheremachine** and **namespace** names from the previous output.

   kubectl describe vspheremachine vks-multinic01-workers-<> -n acme-prod-xxxxx | grep Interfaces -A 11

5. Go back to VCF Automation Organization Portal to verify the VKS cluster status

   a. From **Build & Deploy** tab, go to **Kubernetes** under **Services** in the **acme-prod-xxxxx** namespace

   b. Verify cluster **vks-multinic01** Status is **Available**

   c. Click on cluster **vks-multinic01**

   d. Click **DOWNLOAD KUBECONFIG FILE.** This Kubeconfig is used to connect to the **vks-multinic01**

Task 11: Verify the interfaces on VKS cluster nodes

1. Switch to the **Terminal.**
   Retrieve the VKS cluster SSH password by running the below command:

   kubectl get secret -n acme-prod-xxxxx vks-multinic01-ssh-password -o jsonpath={.data.ssh-passwordkey} | base64 -d

jwzezZs6OnG9RZ8hGLujHzWRok+7Eps1WpRLuKv7j34=holuser@console message appears.

Copy the password part that appears just before holuser@console

2. SSH to the worker node using the IP address recorded in the previous task.

   ```
   ssh vmware-system-user@<Worker node IP>
   ```

   a. Run the command below to verify that interface **eth0** has an assigned IP address while **eth1** does not.

      ```
      ip a
      ```

3. Close the **Terminal**

Lab 17 Creating multi-nic pods on a multi-nic VKS Cluster

Objective and Tasks

You perform the below tasks:

1. Export the vks-multinic01 kubeconfig
2. Create an IPPool custom resource
3. Create a NetworkAttachmentDefinition custom resource
4. Deploy Pods with multiple secondary networks

Task 1: Export the vks-multinic01 kubeconfig

You export the vks-multinic01 kubeconfig in order to connect to the Cluster

1. Switch to the **Terminal**
   a. Export the downloaded Kubeconfig file

      `export KUBECONFIG=/home/holuser/Downloads/vks-multinic01-kubeconfig.yaml`

2. Verify the cluster access by listing the nodes

   `kubectl get nodes`

Task 2: Create an IPPool custom resource

You create an IPPool custom resource for the child subnets

1. Within the `Lab-files/vks-3.6/` directory, review **ippool.yaml**
2. Apply the **ippool.yaml** manifest file to create the **ippools**

```
cd Lab-files/vks-3.6/

kubectl apply -f ippool.yaml
```

3. Verify that **secondary-pod-net-ippool** and **vlan-extension-subnet-ippool** have been created

```
kubectl get ippool
```

Task 3: Create a NetworkAttachmentDefinition custom resource

You create NetworkAttachmentDefinition custom resource for the Pod's secondary interfaces

1. Within the `Lab-files/vks-3.6/` directory, review **nad.yaml**
2. Apply the **nad.yaml** manifest file to create the **NetworkAttachmentDefinition**

```
kubectl apply -f nad.yaml
```

3. Verify that **secondary-pod-nad** and **vlan-extension-nad** have been created in **default** namespace

```
kubectl get network-attachment-definition -n default
```

Task 4: Deploy Pods with multiple secondary networks

You deploy nginx pod with multiple secondary networks

1. Create the namespace for deploying apps

```
kubectl create namespace multi-nic-app-ns
```

2. Label the namespace to remove PSA restrictions

```
kubectl label ns multi-nic-app-ns pod-
security.kubernetes.io/enforce=privileged
```

3. Create image pull secret for Harbor registry

```
kubectl create secret docker-registry harbor-pullsecret \
  --namespace=nginx \
  --docker-server=10.100.6.4 \
  --docker-username=nginx_admin \
  --docker-password='VMware123!VMware123!'
```

4. Within the **Lab-files/vks-3.6/** directory, review **multi-nic-pod.yaml**
5. Apply the **multi-nic-pod.yaml** manifest file to create the **deployment**

```
kubectl apply -f multi-nic-pod.yaml
```

6. Verify that the pod is in running state

```
kubectl get pod -n multi-nic-app-ns
```

7. Verify the interfaces on the pod.
   a. Describe the pod to verify the interfaces in the **Annotations.** Use the exact pod name.

      ```
      kubectl describe pod -n multi-nic-app-ns multi-nic-<>
      ```

   b. Exec to the pod.

      ```
      kubectl exec -it -n multi-nic-app-ns multi-nic-<> -- sh
      ```

   c. Verify the interfaces.

      ```
      ifconfig
      ```

   d. Close the **Terminal** window

Lab 18 Enabling AppArmor Profile on VKS Cluster nodes

Objective and Tasks

You perform the below tasks:

1. Create an AppArmor profile
2. Update the VKS cluster with the AppArmor profile
3. Verify the AppArmor profile on VKS cluster nodes

Task 1: Create an AppArmor profile

You create the AppArmor profile in vSphere Namespace

1. Open a new **Terminal** from the taskbar.
2. List VCF Contexts.

   ```
   vcf context list
   ```

3. Switch to Supervisor admin context

   ```
   vcf context use sup-wld
   ```

   a. If prompted for **Provide Password**, type **`VMware123!VMware123!`** and press Enter.

4. Retrieve the vSphere Namespace name provisioned by VCF Automation

   ```
   kubectl get ns | grep acme-prod
   ```

5. Review the YAML manifest file for creating the apparmor profile `/home/holuser/Lab-files/vks-3.6/apparmor-profile.yaml`

   Update the namespace **acme-prod-xxxxx** as per your environment

6. Apply the YAML manifest file.

   cd Lab-files/vks-3.6

   kubectl apply -f apparmor-profile.yaml

   apparmorprofile.os.kubernetes.vmware.com/vks-multinic01-apparmor created message appears

7. List the apparmorprofile created

   kubectl get apparmorprofile -n acme-prod-xxxxx

Task 2: Update the VKS cluster with the apparmor profile

You update the VKS cluster manifest file to include the AppArmor profile

1. Review the YAML manifest file for updating the VKS Cluster with the apparmor profile **/home/holuser/Lab-files/vks-3.6/vks-multinic01-apparmor.yaml**

   Update the namespace as per your environment **acme-prod-xxxxx**

2. Apply the YAML manifest file.

   cd Lab-files/vks-3.6

   kubectl apply -f vks-multinic01-apparmor.yaml

   cluster.cluster.x-k8s.io/vks-multinic01 configured message appears

3. Wait for the update to be rolled out. Identify the newly created worker node with the AppArmor profile by checking its **AGE**.

   kubectl get machine,vspheremachine -n acme-prod-xxxxx | grep vks-multinic01

   After the old worker nodes are deleted and new ones are provisioned, record the IP addresses of the newly created VKS cluster worker nodes.

Task 3: Verify the AppArmor profile on VKS cluster nodes

1. Retrieve the VKS cluster SSH password by running the below command:

   kubectl get secret -n acme-prod-xxxxx vks-multinic01-ssh-password -o jsonpath={.data.ssh-passwordkey} | base64 -d

   jwzezZs6OnG9RZ8hGLujHzWRok+7Eps1WpRLuKv7j34=holuser@console message appears.

   Copy the password part that appears just before holuser@console

2. SSH to the new worker node using the IP address and retrieved password

ssh vmware-system-user@<Worker node IP>

a. The apprmor profiles are injected to **/etc/apparmor.d** with filename prefix **machine-agent-x**

b. Run the below command to verify the apparmor profile pushed to the VKS cluster node

cat /etc/apparmor.d/machine-agent-0

3. Close the **Terminal** window

Lab 19 Enabling TuneD profile on VKS Cluster nodes

Objective and Tasks

You perform the below tasks:

1. Create a custom TuneD profile
2. Update the VKS cluster with the custom TuneD profile
3. Verify the custom TuneD profile on VKS cluster nodes

Task 1: Create a custom TuneD profile

You create the custom TuneD profile in vSphere Namespace

1. Open a new **Terminal** from the taskbar.
2. List VCF Contexts.

   ```
   vcf context list
   ```

3. Switch to Supervisor admin context

   ```
   vcf context use sup-wld
   ```

   a. If prompted for **Provide Password**, type **`VMware123!VMware123!`** and press Enter.

4. Retrieve the vSphere Namespace name provisioned by VCF Automation

   ```
   kubectl get ns | grep acme-prod
   ```

5. Review the YAML manifest file for creating the tuned profile **/home/holuser/Lab-files/vks-3.6/tuned-profile.yaml**

   Update the namespace **acme-prod-xxxxx** as per your environment

6. Apply the YAML manifest file.

   cd Lab-files/vks-3.6

   kubectl apply -f tuned-profile.yaml

   tunedprofile.os.kubernetes.vmware.com/vks-multinic01-tuned-profile created message appears

7. List the **tunedprofile** created

   kubectl get tunedprofile -n acme-prod-xxxxx

Task 2: Update the VKS cluster with the custom TuneD profile

You update the VKS cluster manifest file to include the tuned profile

1. Review the YAML manifest file for updating the VKS Cluster with the tuned profile **/home/holuser/Lab-files/vks-3.6/vks-multinic01-tuned.yaml**

   Update the namespace **acme-prod-xxxxx** as per your environment

2. Apply the YAML manifest file.

   cd Lab-files/vks-3.6

   kubectl apply -f vks-multinic01-tuned.yaml

   cluster.cluster.x-k8s.io/vks-multinic01 configured message appears

3. Wait for the update to be rolled out. Identify the newly created worker node with the tuned profile by checking its **AGE**.

   kubectl get machine,vspheremachine -n acme-prod-xxxxx | grep vks-multinic01

   After the old worker nodes are deleted and new ones are provisioned, record the IP addresses of the newly created VKS cluster worker nodes.

Task 3: Verify the custom TuneD profile on VKS cluster nodes

1. Retrieve the VKS cluster SSH password by running the below command:

   kubectl get secret -n acme-prod-xxxxx vks-multinic01-ssh-password -o jsonpath={.data.ssh-passwordkey} | base64 -d

   jwzezZs6OnG9RZ8hGLujHzWRok+7Eps1WpRLuKv7j34=holuser@console message appears.

   Copy the password part that appears just before holuser@console

2. SSH to the new worker node using the IP address and retrieved password

ssh vmware-system-user@<Worker node IP>

In case that the new node is using an IP of a recently deleted old worker node, you might get a warning that remote host's identification has changed. Remove the stale SSH host key: **ssh-keygen -f '/home/holuser/.ssh/known_hosts' -R 'Worker Node IP'**

a. Verify the current active tuned profile on the node with the below command

   sudo tuned-adm active

   Current active profile: custom-tuned-profile message appears

b. Custom Tuned profile is injected into the node at **/etc/tuned/<profile-name>/tuned.conf**

   sudo cat /etc/tuned/custom-tuned-profile/tuned.conf

3. Close the **Terminal** window

Lab 20 Monitoring VKS Clusters from VCF Operations

Objective and Tasks

You perform the below tasks to verify monitoring for the VKS Cluster

1. Verify Metrics Aggregator Service status
2. Verify Prometheus Add-on status
3. Verify Telegraf Add-on status
4. Monitoring VKS Cluster from VCF Operations
5. Monitoring VKS Cluster from VCF Automation
6. Monitoring VKS Cluster from VKS Cluster Management

Task 1: Verify Metrics Aggregator Service status

You validate the Metrics Aggregator Service installation status.

1. Log in to the VCF Automation Provider Management Console.
    a. In **Firefox**, open a new browser tab and select **Region A > VCF Automation** from the bookmark
    b. For **Organization name**, enter **`system`**.
    c. Click **Continue**.
    d. Log in to the VCF Automation Provider Management Console.
        - **Username**: **`admin`**
        - **Password**: **`VMware123!VMware123!`**
2. Verify the Metrics Aggregator Service installation status.

a. From the navigation pane, click **Service Management**.

b. Locate the Metrics Aggregator Service, click the **Details** button.

Review the Metrics Aggregator Service status, version, regions, and VCF Service Components resources.

Task 2: Verify Monitoring status on VKS cluster

You verify the automated-monitoring label on the VKS cluster

1. Open a new **Terminal** from the taskbar
2. List the contexts

   vcf context list

3. Switch to the Sueprvisor context

   vcf context use sup-wld

4. Get the name of the namespace where VKS cluster is deployed

   kubectl get ns | grep acme-prod

5. List the clusters

   kubectl get cluster -n acme-prod-jbv6z

6. Verify that the label `addons.kubernetes.vmware.com/automated-monitoring` is present in the cluster

   kubectl get cluster -n acme-prod-jbv6z vks-mt-app01 -o yaml | grep monitoring

7. Close the **Terminal**

Task 3: Verify Prometheus and Telegraf Add-ons status

Prometheus and Telegraf Add-ons are configured by default using the default Repository on VKS Cluster Management.

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **acme-org**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - **Username**: **admin**
      - **Password**: **VMware123!VMware123!**

2. Click the **Manage & Govern** tab
3. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
4. Click on VKS cluster **vks-mt-app01**
5. Click on the **Add-ons** tab
    a. Under **Repositories**, verify that the below default Repository is present.

    Name: default-addon-repo-install

    URL: depot.kube-system.svc/vcf/vks-standard-packages/ga/3.6.0-20260211/vks-standard-packages:3.6.0-20260211

    Status: Succeeded

    b. Under **Installed Add-ons**, verify that **Prometheus** and **Telegraf** Add-ons are present and status is green

Task 4: Monitoring VKS Cluster from VCF Operations

You monitor VKS clusters from VCF Operations

1. On the **Firebox** bookmarks bar, click **VCF Operations**
    a. Log in to the VCF Operations portal
        - Login Method: Local Account
        - **Username**: **`admin`**
        - **Password**: **`VMware123!VMware123!`**
2. Monitor VKS Cluster

    On the top search bar, search for **`vks-mt-app01`** Cluster and click on it.

    a. To review the VKS Cluster Health Summary, click **Summary** tab. View total number of objects, resource utilization and alerts.

    b. To review the VKS Cluster topology, click **Topology** tab. It shows the relationships between all objects.

    c. To review VKS Cluster Metrics, click **Metrics** tab. Expand **Metrics** and then expand **CPU** and **Memory**. Initially, this will be blank. Double-click on metrics such as **Usage (cores)** under **CPU, Usage (GB)** under **Memory**, to generate the data.
3. Viewing VKS Cluster Dashboards

    In the **Operate** tab, navigate to **Dashboards**. Expand the **All** folder and go to **vSphere Supervisor > VKS Cluster.**

a. In **VKS Clusters** tile, click on the **vks-mt-app01** row. Don't click on the **vks-mt-app01** itself.

b. In the **Control Plane Nodes** tile, click on each row to get the **ETCD, API Server and Controller Manager details** on the respective Nodes.

c. Verify the kubernetes object resources under **Relevant Properties** tile

Task 5: Monitoring VKS Cluster from VCF Automation

Monitor VKS clusters from VCF Automation using your Organization User account

1. On the **Firefox** bookmarks bar, click **VCF Automation**
   a. Under **Organization name**, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Login using Local user
      - **Username**: **`admin`**
      - **Password**: **`VMware123!VMware123!`**
2. In the Organization portal, click on **Build & Deploy** tab.
   a. Click **Kubernetes** under **Services**.
   b. Click **vks-mt-app01** and go to **Monitor** tab
   c. Scroll down. Under opencart, click **my-open-cart-app** to view the resource utilization
   d. Scroll further down. Click on the VKS Cluster nodes to view the resource utilization

Task 6: Monitoring VKS Cluster from VKS Cluster Management

Monitor VKS clusters from VKS Cluster Management using your Organization Admin account

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation**
   a. Under Organization name, enter
   b. Log in to the VCF Automation Organization Portal.
      - Login using Local user
      - User name: **`admin`**
      - Password: **`VMware123!VMware123!`**
2. In the Organization portal, click the **Manage & Govern** tab.
   a. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.

3. Under **Clusters**, review the clusters dashboard table.
    a. **Health**: Displays a green **Healthy** badge
    b. **Status**: Displays **Ready** status
    c. **Version**: Displays running Kubernetes version on the cluster
4. Deep Dive into Cluster Health, Agents, and Extensions

    From the all clusters list, click on **vks-mt-app01**
5. On the **Overview** tab, review the following metadata:
    a. Verify requested and allocatable CPU and memory resources
    b. Verify core Kubernetes control plane items like the controller-manager, etcd-0, kube-apiserver, and scheduler are fully functional.
6. On the Nodes tab, review the following:

    Verify the Nodes Health **Status**, **kubelet version**, **Requested/Allocatable CPU** and **Requested/Allocatable memory**
7. On the **Namespaces** tab, review the following:

    Click on **opencart** namespace and verify the resources
8. On the Resources tab, review the following:

    Verify all resource objects **Health**, **Namespace** and **Labels**.
9. On the **Add-ons** tab, review the following:

    Installed Add-ons: Verify all installed Add-ons in the cluster

    Available Add-ons: Verify all available Add-ons for the cluster

    Repositories: Verify default repository created by vksm to pull Add-on package images
10. Data Protection tab gets enabled when Data Protection is enabled on the cluster. This will be enabled in a later lab.

Lab 21 Backing Up Applications Using Velero

Objective and Tasks

You configure data protection, establish an S3 backup target, create an application-scoped backup of an active namespace using the VCF Automation user interface

1. Enable Data Protection on the Cluster
2. Configure S3 Account Credentials
3. Map the S3 Target Backup Location
4. Perform Backup Prechecks
5. Create a Targeted Namespace Backup
6. Perform Backup Postchecks

Task 1: Enable Data Protection on the Cluster

You enable data protection in **vks-mt-app01** using VKS Cluster Management from VCF Organization Portal

1. On the **Firefox** bookmarks bar, click **Region A > VCF Automation.**
   a. Under Organization name, enter **`acme-org`**
   b. Log in to the VCF Automation Organization Portal.
      - Log in with local account
      - **Username**: **`admin`**
      - **Password**: **`VMware123!VMware123!`**
2. Click the **Manage & Govern** tab

3. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
4. Click on VKS cluster **vks-mt-app01**
5. Locate the Data Protection card at the bottom and click **ENABLE DATA PROTECTION**
   a. Review the details and then click **ENABLE**
   b. Wait for data protection to be enabled. Below message appears.

      No target location is configured for this cluster. Visit 'Configuration > Target' location to create one.

Task 2: Configure S3 Account Credentials

You create access keys required to authenticate against your S3 bucket endpoint

1. Within the Kubernetes Management panel, navigate to the Configurations menu.
2. Click the Credentials sub-tab
3. Click the **CREATE CREDENTIAL** button and select **AWS S3 or S3-compatible**
4. Fill out the Create account credential

Parameter	Value
Credential Name	minio-s3-credential
Access key ID	minioadmin
Secret Access key	VMware123!

5. Click **CREATE**
6. Verify that **s3-object-store** credential status shows **Created**

Task 3: Map the S3 Target Backup Location

You create the target bucket where Velero stores cluster metadata and data replication.

1. Switch to the Target Locations sub-tab under Configurations.
2. Click **CREATE TARGET LOCATION** button and select **AWS S3 or S3-compatible**
3. Fill out the target endpoints configuration fields

Parameter	Value

Account credential	minio-s3-credential
AWS S3 or S3-compatible URL	http://10.1.10.205:9000
Bucket	velero-backups
Region	minio

4. Click **NEXT**
5. Under **Clusters**, click **SELECT CLUSTERS**
   a. Select **vks-mt-app01** and click **SELECT**
6. Click **NEXT**
7. In **Name** field, enter **minio-s3-bucket**
8. Click **CREATE**

Task 4: Perform Backup Prechecks

You perform the prechecks of Velero server and application health before taking the backup

1. Switch to the **Terminal**
   a. Export the downloaded Kubeconfig file

      ```
      export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml
      ```

2. Verify Velero Server status in the cluster
   a. Run the below command

      ```
      kubectl get all -n velero
      ```

      All pods are in **Running** state.
3. Verify S3 bucket integration status
   a. Run the below command

      ```
      velero backup-location get
      ```

      minio-s3-bucket Phase is **Available**
4. Verify the Application Health
   a. Run the below commands

      ```
      kubectl get all -n opencart
      ```

Deployment status is **READY 1/1**

`kubectl get pvc -n opencart`

open-cart-pvc is in **Bound** state

Task 5: Create a Targeted Namespace Backup

You capture a point-in-time snapshot of your target application namespace using Velero

1. Return to VCF Automation Organization Portal. Click the **Manage & Govern** tab.
2. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
3. Click on VKS cluster **vks-mt-app01**
4. Locate the **Data Protection** card at the bottom and click **CREATE BACKUP**
5. In the **What Kubernetes resources to backup** section, choose **Back up selected namespaces**.
6. Check the box corresponding to **opencart** namespace
7. Click **Next** to move to the **How to backup volumes** section
8. Configure the volume capturing properties
    a. Select **Use FSB Opt-in approach**
    b. Check **Use CSI snapshot backup for persistent volumes**

Check **Move CSI snapshot data to backup target location**. This is checked by default.

9. Click **NEXT** to move to **Where to store the backup**
    a. Select the **Target Location minio-s3-bucket** form drop-down
10. Click **NEXT** and select Schedule type **NOW**
11. ClicK **NEXT** and keep **Retention (days) 30**
12. Click **NEXT** and enter the name **velero-backup-vksm**
13. Click **CREATE**
14. Wait for Backup **velero-backup-vksm** status to reach **100%**
15. Verify the backup on Minio Console
    a. On the **Firefox** bookmarks bar, click **Region A > MinIO Console.**
    b. Log in to the Portal.
        - **Username**: **`minioadmin`**
        - **Password**: **`VMware123!`**

c. In the Home page, under **velero-backups,** click on the folder available.

   it leads to **backups** and **kopia** folders

d. Further click on the **backups** folder to view **velero-backup-vksm**

e. Further click on the **velero-backup-vksm** folder to view content of the backup

Task 6: Perform Backup Postchecks

You perform the postchecks of the backup

1. Switch to the **Terminal**

   a. Export the downloaded Kubeconfig file

      ```
      export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml
      ```

2. Verify Velero backup status

   a. Run the below command

      ```
      velero backup get
      ```

      Backup **Status** should be in Completed state.

3. Run velero backup describe to verify how many items were backed up, confirm which namespaces were included.

   a. Run the below command

      ```
      velero backup describe velero-backup-vksm
      ```

4. Verify the backuprepositories

   Run the below command

   ```
   kubectl get backuprepositories -n velero
   ```

5. Verify the dataupload status

   Run the below command

   ```
   kubectl get dataupload -n velero
   ```

Lab 22 Restoring Applications Using Velero

Objective and Tasks

You simulate a failure, and perform a full restore using the VCF Automation user interface

1. Simulate an Application Failure
2. Execute an Application Restore
3. Verify Application Recovery

Task 1: Simulate an Application Failure

you delete all resources in the namespace to simulate application failure

1. Switch to the **Terminal**
   a. Export the downloaded Kubeconfig file

      `export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml`
2. Delete the opencart application
   a. Change directory **`cd Lab-files/opencart/`**
   b. Run the below command

      `kubectl delete -f opencart.yaml`
3. Delete the opencart application Load Balancer IP
   a. Run the below command

      `kubectl delete -f opencart-lb.yaml`
4. Delete the opencart namespace

a. Run the below commands

```
kubectl delete ns opencart
```

b. Verify that pvc has been deleted

```
kubectl get pv | grep opencart
```

You should not see any matching pv with the above command

Task 2: Execute an Application Restore

You restore an application from a point-in-time snapshot of your target application namespace using Velero

1. Return to VCF Automation Organization Portal. Click the **Manage & Govern** tab.
2. In the left navigation tree, scroll to the bottom and select **Kubernetes Management**.
3. Click on VKS cluster **vks-mt-app01**
4. Select the **Data Protection** tab
5. Look at the historical inventory table listing completed backups and select **velero-backup-vksm**
6. Click on the three dots icon next to **velero-backup-vksm** entry and select **RESTORE**
7. In the **What Kubernetes resources to restore** section, Change the selection from **Restore the entire backup** to **Restore selected namespace**
8. In the namespace inventory grid list, select the checkbox next to **opencart**
9. In the **What volumes to restore** section, select the checkbox for **Restore persistent volume using CSI snapshot**
10. Click **NEXT**
11. In the **Name and create** section, enter **`velero-restore-vksm`**
12. Click **RESTORE**
13. Wait for Restore **velero-restore-vksm** status to reach **100%**
14. Verify the restore on Minio Console
    a. On the **Firefox** bookmarks bar, click **Region A > MinIO Console.**
    b. Log in to the Portal.
       - **Username**: **`minioadmin`**
       - **Password**: **`VMware123!`**
    c. In the Home page, under **velero-backups,** click on the folder available.

you should see the third folder **restores**

d. Further click on the **restores** folder to view the metadata and logs for the restore operation.

Task 3: Verify Application Recovery

You perform the postchecks of the restore and application health check.

1. Switch to the **Terminal**

   a. Export the downloaded Kubeconfig file

      export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml

2. Verify Velero backup status

   a. Run the below command

      velero restore get

      Restore **Status** should be in **Completed** state.

3. Run velero restore describe to verify how many items were restored, confirm which namespaces were included.

   a. Run the below command

      velero restore describe velero-restore-vksm

4. Verify the datadownload status

   Run the below command

   kubectl get datadownload -n velero

5. Verify the application health

   Run the below commands

   kubectl get ns opencart

   kubectl get pod,pvc -n opencart

   kubectl get pv | grep opencart

   kubectl get svc -n opencart | grep Load

Lab 23 Updating a VKS Cluster using a Custom Rolling Update Strategy

Objective and Tasks

You perform the below tasks to update a VKS Cluster

1. Registering a New Version of the VKS
2. Upgrading the VKS Version
3. Updating the VKS Cluster Using VCF CLI
4. Generate VKS Cluster Support Bundle
5. [Appendix] Workaround for failed compatibility check due to VMwareSystemProperties validation on Non-VKS content library

Task 1: Customize VKS Cluster Rolling Update Strategy

You inspect the active MachineDeployment rollout configuration , export the cluster manifest, modify the node pool's rolling update parameters, and verify that Cluster API (CAPI) reconciles the change.

1. Open a new **Terminal** from the taskbar.
2. Switch to the Supervisor admin context

   ```
   vcf context use sup-wld
   ```

3. Get the namespace where vks-mt-app01 is deployed

   ```
   kubectl get ns | grep acme-prod
   ```

4. Run the below command to locate the MachineDeployment associated with the vks-mt-app01 cluster:

kubectl get machinedeployment -n acme-prod-5zn65 | grep vks-mt-app01

5. Using the MachineDeployment name returned from the output, query the current .spec.rollout configuration using jsonpath:

   kubectl get machinedeployment -n acme-prod-5zn65 vks-mt-app01-vks-mt-app01-np-01-47lb4 -o jsonpath='{.spec.rollout}'

   Expected Output -
   `{"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":0},"type":"RollingUpdate"}}`

   Notice that maxSurge is currently set to 1 (the default value)

6. Export the live **vks-mt-app01** Cluster specification from the Supervisor namespace into a local YAML file:

   kubectl get cluster -n acme-prod-5zn65 vks-mt-app01 -o yaml > vks-mt-app01.yaml

7. Modify the Rolling Update Strategy

   Open vks-mt-app01.yaml in a text editor.

   Navigate to `spec.topology.workers.machineDeployments` and update the `rollout.strategy` to have maxSurge 3.

   ```
   workers:
     machineDeployments:
     - class: node-pool
       metadata:
         annotations:
           run.tanzu.vmware.com/resolve-os-image: os-name=photon, content-library=cl-44b09f305973fe655
       name: vks-mt-app01-np-01
       replicas: 3
       rollout:
         strategy:
           type: RollingUpdate
           rollingUpdate:
             maxSurge: 3
             maxUnavailable: 0
   ```

8. Apply the modified YAML manifest back to the Supervisor cluster to reconcile the changes:

   kubectl apply -f vks-mt-app01.yaml

Expected Output - `Warning: resource clusters/vks-mt-app01 is missing the kubectl.kubernetes.io/last-applied-configuration annotation...`
`cluster.cluster.x-k8s.io/vks-mt-app01 configured`

Note: The warning message is expected when applying resources exported directly from kubectl get -o yaml

9. Verify that CAPI has propagated the updated rollout configuration down to the underlying MachineDeployment

   `kubectl get machinedeployment -n acme-prod-5zn65 vks-mt-app01-vks-mt-app01-np-01-47lb4 -o jsonpath='{.spec.rollout}'`

   Expected Output -
   `{"strategy":{"rollingUpdate":{"maxSurge":2,"maxUnavailable":0},"type":"RollingUpdate"}}`

Task 2: Registering a New Version of the VKS

You register a new version of VKS from vSphere Client

1. Log in to the vSphere Client.
   a. On the taskbar, click the **Firefox** icon.
   b. Click **Region A** > **vc-wld01-a** on the bookmarks bar to open the vSphere Client.
   c. Log in.
      - **Username**: **`administrator@wld.sso`**
      - **Password**: **`VMware123!VMware123!`**
2. In the vSphere Client, select **Supervisor Management** from the menu in the top-left corner.
3. On the **Supervisor Management** page, click the **Services** tab.
4. Go to the **Kubernetes Service** tile

   Click **Active Versions**

   a. Verify that current version is **3.7.0+v1.36.** Click **Close**
   b. Click **ACTIONS**
   c. Click **Add New version**
   d. On the Register Service page, select **UPLOAD.**
   e. Navigate to the **/home/holuser/Lab-files/service-yaml** folder

f. Select **vsphere-kubernetes-service-3.7.1+v1.36.yaml**

g. Click **Select**.

YAML was uploaded successfully. Note: YAML content is not verified and could fail during installation into a Supervisor message appears

h. Click **FINISH**.

Task 3: Upgrading the VKS Version

You upgrade VKS version from vSphere Client

1. Click **ACTIONS** from the **Kubernetes Service** tile.
2. Click **Manage Service**
3. On the manage wizard, select the install version **3.7.1+v1.36**
4. Select the Supervisor **supervisor-wld-a**
5. Click **NEXT**.

   The following Compatibility messages appear on the wizard.

   Validation completed succesfully. Review messages below.
6. Click **Next**
7. In the Review Page, leave **YAML Service Config (optional)** as it is.
8. Click Finish

   New 'Kubernetes Service' version [3.7.1+v1.36] is available for use on your Supervisors message appears

Task 4: Updating the VKS Cluster Using VCF CLI

You update the vks-mt-app01 using VCF CLI

1. Open a new **Terminal** from the taskbar.
2. Switch to the Supervisor admin context

   ```
   vcf context use sup-wld
   ```
3. Verify that vKR **v1.36.2---vmware.2-vkr.3** has **READY** and **COMPATIBLE** fields set to **TRUE**

   New vKR sync may take some time. Wait for the **READY** and **COMPATIBLE** status to become **TRUE**

   ```
   kubectl get kr | grep -iE "NAME|1.36"
   ```

4. Verify that clusterclass **builtin-generic-v3.7.1** is available in the namespace **vmware-system-vks-public** and **VARIABLES READY** is set to **TRUE**

   ```
   kubectl get clusterclass -n vmware-system-vks-public | grep v3.7
   ```

5. Get the namespace where **vks-mt-app01** is deployed

   kubectl get ns | grep acme-prod

6. Verify that **READY** status for all the **ClusterAddon** objects is **True**

   kubectl get ClusterAddon -n acme-prod-xxxx | grep vks-mt-app01

7. Get the cluster details

   kubectl get cluster vks-mt-app01 -n acme-prod-xxxx

   VERSION v1.35.5+vmware.1

8. Get the available upgrades for cluster **vks-mt-app01**

   vcf cluster available-upgrades get vks-mt-app01 -n acme-prod-xxxx

9. Upgrade the cluster **vks-mt-app01** to the target vkr **v1.36.2---vmware.2-vkr.3**

   vcf cluster upgrade vks-mt-app01 -n acme-prod-5zn65 --kr v1.36.2---vmware.2-vkr.3

   Enter **y** when prompted `Upgrading workload cluster 'vks-mt-app01' to kubernetes version 'v1.36.2+vmware.2-vkr.3', tkr version 'v1.36.2+vmware.2-vkr.3'. Are you sure? [y/N]:`

   If you get below error: `Error: unable to patch kubernetes version to cluster: unable to patch the cluster object: error while applying patch for "&TypeMeta{Kind:,APIVersion:,}" acme-prod-5zn65/vks-mt-app01: admission webhook "capi.validating.tanzukubernetescluster.run.tanzu.vmware.com" denied the request: update cannot be initiated since addon precheck before upgrade has failed: cluster cannot be upgraded: clusteraddon: vks-mt-app01-antrea, unable to infer latest addon install expected to find at least one, but did not (details: all=44 -> after-addon-name-filter=8 -> after-namespace-filter=4 -> after-install-constraints-filter=0) failed to select item`

   Apply the workaround and rerun the upgrade command.

   kubectl apply -f ~/Lab-files/vks-3.7/vks-mt-app01-addon-fix.yaml

```
addoninstall.addons.kubernetes.vmware.com/vks-mt-app01-
antrea created
addoninstall.addons.kubernetes.vmware.com/vks-mt-app01-
prometheus created
```

10. Open another **Terminal** to connect to **vks-mt-app01** using VCF CLI

    `export KUBECONFIG=/home/holuser/Downloads/vks-mt-app01-kubeconfig.yaml`

    Watch the cluster node upgrade. Control plane nodes will upgrade sequentially first, followed by the node pool nodes. During the node pool upgrade, two nodes will be provisioned simultaneously based on the configured MaxSurge value of 3

    `watch kubectl get nodes`

    Version will show **v1.36.2+vmware.2** for the upgraded nodes. Keep the watch window open till all control-plane nodes and worker nodes are recreated on version **v1.36.2+vmware.2** and their status turns **Ready**

    Proceed to the next step. Enter Control + C to exit the **watch** window once upgrade completes

11. Login to vSphere Client and monitor new node creation

    You log in to the **vc-wld01-a.site-a.vcf.lab** vCenter.

    From the bookmarks toolbar, select Region A > **vc-wld01-a**

    Log in by entering **`administrator@wld.sso`** as the username and **`VMware123!VMware123!`** as the password.

    From the vSphere Client main menu, select Inventory. and go to **Hosts and Cluster** View

    Go to the namespace **acme-prod-xxx** and view the nodes of the cluster **vks-mt-app01**

12. On the first **Terminal** where the upgrade was initiated, wait for upgrade to complete.

    `Cluster 'vks-mt-app01' successfully upgraded to kubernetes version 'v1.36.2+vmware.2-vkr.3'` messages appears when the cluster upgrade completes successfully,

13. If the upgrade fails or times out, check for any alert on the Supervisor Cluster and fix it.

    Rerun the upgrade with **--force** option

    `vcf cluster upgrade vks-mt-app01 -n acme-prod-5zn65 --kr v1.36.2---vmware.2-vkr.3 --force`

Answer Key

No table of contents entries found.